Chương 12

Lập luận

Trong các chương trước, chúng ta đã xem xét LLM cho các ứng dụng đa dụng, trong đó mô hình sinh ra phản hồi dựa trên truy vấn đầu vào. Những mô hình này có thể xử lý một cách tự nhiên rất nhiều loại nhiệm vụ, chẳng hạn dịch máy và hỏi đáp. Vì vậy, nhiều bài toán NLP có thể được phát biểu thành các nhiệm vụ sinh văn bản và giải quyết bằng LLM. Vượt ra ngoài việc sinh văn bản đơn thuần, bước tiếp theo tự nhiên là phát triển những mô hình có khả năng suy nghĩ sâu hơn để giải các bài toán phức tạp hơn. Đặc biệt, ta mong muốn LLM có thể sinh ra các bước suy nghĩ trung gian và nhờ đó thể hiện khả năng lập luận (*reasoning*).

Lập luận từ lâu đã được coi là một trong những bài toán khó nhất của AI. Mặc dù cộng đồng NLP đã cố gắng giải quyết nó bằng các phương pháp như phân tích câu thành các dạng logic (*logical forms*), những nỗ lực này chủ yếu dựa vào việc xây dựng mô hình lập luận từ một chuỗi các mô-đun nối tiếp nhau theo kiểu dây chuyền (*pipeline*), vốn thường dễ bị lỗi lan truyền dây chuyền (*cascading errors*). Ngược lại, LLM cung cấp một khung tổng quát hơn cho lập luận, trong đó con đường suy nghĩ có thể được biểu đạt tường minh dưới dạng văn bản, và vì thế ta có thể cho LLM lập luận ngay trong quá trình sinh văn bản. Dù lập luận có thể được xem là một năng lực trỗi dậy (*emergent ability*) xuất hiện khi mở rộng quy mô LLM, việc khơi dậy năng lực này đòi hỏi những chiến lược cụ thể, bao gồm prompt chuỗi suy nghĩ (CoT), mở rộng quy mô lúc suy luận (*inference-time scaling*) và huấn luyện LLM để lập luận.

Trong chương này, chúng ta sẽ thảo luận các phương pháp cải thiện năng lực lập luận của LLM. Ta bắt đầu bằng việc giới thiệu các khái niệm

cơ bản liên quan đến lập luận. Tiếp theo, ta trình bày các kỹ thuật để phát triển mô hình lập luận (*reasoning models*). Vì kỹ thuật prompt và các phương pháp CoT đã được bàn kỹ trong các chương trước, ở đây ta tập trung vào hai hướng nghiên cứu: 1) mở rộng lượng tính toán lúc suy luận (*inference-time compute*) và 2) học để lập luận (*learning to reason*). Cuối cùng, ta tổng kết chương bằng một số nhận xét.

Chú giải của người dịch: "Lập luận" và "suy luận" trong chương này

Trong chương này, *reasoning* được dịch là **lập luận**: quá trình đi từng bước từ dữ kiện đến kết luận, giống như khi bạn trình bày lời giải một bài toán. Còn *inference* được dịch là **suy luận**, với nghĩa kỹ thuật là *chạy mô hình đã huấn luyện để sinh ra đầu ra* (đối lập với huấn luyện). Do đó "lúc suy luận" (*inference time*) nghĩa là "lúc mô hình đang được dùng để trả lời", không phải lúc nó đang được huấn luyện.

12.1 Bối cảnh và các khái niệm

Ở đây chúng ta định nghĩa bài toán lập luận và thảo luận các khái niệm liên quan.

12.1.1 Lập luận là gì?

Trong lịch sử, nghiên cứu trí tuệ nhân tạo đã phân loại lập luận thành ba dạng logic chính: diễn dịch (*deduction*) (rút ra kết luận cụ thể từ các tiền đề đã biết, chẳng hạn các chứng minh toán học), quy nạp (*induction*) (suy ra các quy tắc hay khuôn mẫu tổng quát từ những ví dụ cụ thể) và giả định (suy ra lời giải thích tốt nhất) (*abduction*) (tìm lời giải thích hợp lý nhất cho một tập các quan sát) [Hartshorne và Weiss, 1931]. Trong một thời gian dài, dòng nghiên cứu chủ đạo của học máy và NLP chủ yếu tập trung vào các nhiệm vụ quy nạp. Chẳng hạn, các ứng dụng NLP kinh điển như phân loại văn bản và nhận dạng thực thể có tên có thể được xem là các nhiệm vụ quy nạp: mô hình được huấn luyện để khái quát hóa các hàm ánh xạ từ những tập dữ liệu lớn đã được gán nhãn.

Chú giải của người dịch: Ba kiểu lập luận qua ví dụ

- **Diễn dịch**: "Mọi số chia hết cho 6 đều chia hết cho 3; 72 chia hết cho 6; vậy 72 chia hết cho 3." Nếu tiền đề đúng thì kết luận *chắc chắn* đúng.
- **Quy nạp**: "Tôi thấy 1000 con quạ đều màu đen, vậy có lẽ mọi con quạ đều màu đen." Kết luận tổng quát rút ra từ các ví dụ, có thể sai. Học máy truyền thống chính là quy nạp: từ nhiều ví dụ có nhãn, mô hình học ra một quy luật chung.
- **Giả định** (*abduction*): "Sáng nay sân ướt; lời giải thích hợp lý nhất là đêm qua trời mưa." Ta chọn giả thuyết giải thích tốt nhất cho điều quan sát được, giống như bác sĩ chẩn đoán bệnh từ triệu chứng.

Tuy nhiên, cùng với sự phát triển của LLM, các nhà nghiên cứu đang cố gắng giải quyết những bài toán lập luận khó hơn, như toán học phức tạp, lập kế hoạch kiểu tác tử (*agentic planning*) và khám phá khoa học, vốn thường đòi hỏi lập luận diễn dịch và giả định chặt chẽ. Nghiên cứu LLM hiện đại đã cho thấy mục tiêu tiền huấn luyện tiêu chuẩn (tức là dự đoán token tiếp theo trên những kho ngữ liệu văn bản khổng lồ) đủ mạnh để giúp mô hình nội tại hóa tri thức về ngôn ngữ và thế giới, kể cả các năng lực lập luận tiềm ẩn. Tuy nhiên, thách thức chính là lập luận không được mô hình hóa một cách tường minh trong giai đoạn tiền huấn luyện. Kết quả là, khi gặp một bài toán phức tạp, LLM cố gắng đưa ra đáp án cuối cùng ngay lập tức, thay vì thực hiện một quá trình suy diễn logic từng bước.

Ví dụ, xét một bài toán đố đơn giản:

> Nếu tuổi của John gấp đôi tuổi của Mary, và 3 năm nữa Mary sẽ 10 tuổi, thì hiện nay John bao nhiêu tuổi?

Một LLM nền có thể đưa ra một phỏng đoán sai (ví dụ 20) chỉ đơn giản bằng cách liên kết các từ khóa "gấp đôi" và "10". Ngược lại, một LLM có năng lực lập luận có thể sinh ra chuỗi logic một cách tường minh trước khi trả lời:

> Hiện nay Mary $10 - 3 = 7$ tuổi. Do đó, John $7 \times 2 = 14$ tuổi.

Để giải các nhiệm vụ diễn dịch và giả định phức tạp bằng LLM, những

năng lực lập luận tiềm ẩn này phải được kích hoạt một cách tường minh, dù là thông qua các kỹ thuật prompt như CoT hay thông qua các mô thức hậu huấn luyện (*post-training*) tiên tiến như học tăng cường.

Để hiểu rõ hơn bài toán lập luận, ta có thể nhìn nhận năng lực của LLM qua thuyết quá trình kép (*dual-process theory*) trong tâm lý học nhận thức: tư duy Hệ thống 1 và Hệ thống 2 [Kahneman, 2011].

- **Hệ thống 1 (tư duy trực giác).** Trong nhận thức của con người, Hệ thống 1 hoạt động tự động và nhanh chóng, hầu như không tốn công sức. Trong bối cảnh LLM, tư duy Hệ thống 1 tương ứng với mô thức trong đó mô hình đưa ra trực tiếp đáp án cuối cùng chỉ trong một lượt. Chế độ này hiệu quả và nhìn chung đủ dùng cho các nhiệm vụ quy nạp, như hỏi đáp đơn giản. Tuy nhiên, tư duy Hệ thống 1 dễ mắc các bước nhảy logic và ảo giác khi đối mặt với những bài toán phức tạp nhiều bước.
- **Hệ thống 2 (tư duy cân nhắc).** Chế độ này chậm và có ý thức, đòi hỏi suy diễn logic từng bước. Với LLM, tư duy Hệ thống 2 được kích hoạt khi mô hình được chỉ dẫn phân bổ thêm tài nguyên tính toán lúc suy luận để sinh ra một cách tường minh các bước lập luận trung gian. Bằng cách biểu diễn quá trình suy nghĩ dưới dạng văn bản, LLM chia nhỏ các nhiệm vụ phức tạp, kiểm chứng các kết luận trung gian và duy trì một chuỗi logic mạch lạc trước khi đi đến đáp án cuối cùng.

Chú giải của người dịch: Hệ thống 1 và Hệ thống 2 trong đời thường

Khi được hỏi "$2+2$ bằng mấy?", bạn trả lời ngay "4" mà không cần nghĩ: đó là Hệ thống 1. Khi được hỏi "17×24 bằng mấy?", bạn phải đặt tính, nhân từng hàng, cộng lại ($17 \times 24 = 17 \times 20 + 17 \times 4 = 340 + 68 = 408$): đó là Hệ thống 2. LLM "trả lời ngay" giống như cố đoán 17×24 mà không đặt tính; LLM "lập luận" giống như viết ra từng dòng tính toán trên giấy nháp.

Đối với lập luận của LLM, mục tiêu cốt lõi là kích hoạt năng lực Hệ thống 2. Để hiểu điều này, ta phải nhìn xa hơn logic ký hiệu truyền thống. Trong AI cổ điển, lập luận thường được định nghĩa là việc áp dụng các quy tắc logic được lập trình cứng lên các cơ sở tri thức (*knowledge bases*) có cấu trúc. Tuy nhiên, với LLM, lập luận là một quá trình sinh văn bản

nội tại. Nó có thể được định nghĩa là khả năng sinh ra một chuỗi mạch lạc các bước trung gian để đi đến đáp án cuối cùng. Theo một nghĩa nào đó, lập luận trong LLM là một cơ chế phân bổ động thêm lượng tính toán suy luận cho những bài toán khó hơn. Rốt cuộc, lập luận đối với một LLM là năng lực mã hóa tư duy Hệ thống 2 tiềm ẩn thành một chuỗi các bước suy nghĩ (*thinking steps*) (các bước lập luận (*reasoning steps*)).

12.1.2 Vì sao lập luận quan trọng

Lập luận quan trọng trong nghiên cứu NLP vì nó được ứng dụng trong nhiều lĩnh vực đầy thách thức:

- **Toán học.** Giải toán đòi hỏi logic chặt chẽ. Một mô hình không thể giải bài toán một cách đáng tin cậy chỉ bằng cách liên kết các con số và từ khóa trong câu hỏi với những đáp án đã thấy ở các ví dụ tương tự. Thay vào đó, nó cần áp dụng các quy tắc toán học từng bước để suy ra kết quả đúng.
- **Lập trình.** Viết phần mềm khó hơn nhiều so với việc chỉ dự đoán từ tiếp theo. Mô hình phải sinh mã dựa trên các yêu cầu cụ thể của người dùng và logic chặt chẽ của mã. Đặc biệt khi giải quyết những bài toán phức tạp, việc chuyển các yêu cầu trừu tượng thành mã chạy được đòi hỏi lập luận từng bước.
- **Lập kế hoạch kiểu tác tử.** Khi một LLM được dùng như một tác tử (*agent*) tự hành, nó phải chia các mục tiêu lớn, phức tạp thành những bước nhỏ, có thể thực hiện được. Nó cũng cần lên kế hoạch trước và điều chỉnh hành động nếu mọi việc không như ý.
- **Khám phá khoa học.** Hỗ trợ các nhà khoa học đòi hỏi phải xem xét dữ liệu, đề xuất các giả thuyết mới và kiểm chứng chúng một cách cẩn thận. Việc này dựa rất nhiều vào suy diễn và giả định logic từng bước.

Ngoài việc giải được những bài toán khó hơn, sinh ra các bước lập luận còn mang lại những lợi ích thực tiễn. Quan trọng nhất, nó làm cho mô hình trở nên diễn giải được (*interpretable*). Vì mô hình viết ra quá trình suy nghĩ của nó dưới dạng văn bản, con người có thể đọc và kiểm tra logic. Họ thậm chí có thể tìm ra chính xác chỗ nào đã mắc lỗi. Điều

này giúp hệ thống dễ kiểm chứng và dễ tin cậy hơn nhiều. Ngoài ra, lập luận từng bước giúp giảm ảo giác. Thay vì đoán bừa khi gặp câu hỏi khó, mô hình đặt đáp án cuối cùng của mình trên nền tảng các bước logic của chính nó, khiến đầu ra đáng tin cậy hơn nhiều.

12.1.3 Từ LLM đến mô hình lập luận

Trước khi LLM chiếm ưu thế, lập luận trong AI chủ yếu xoay quanh các quy tắc và logic chặt chẽ. Các nhà nghiên cứu dùng những phương pháp như AI ký hiệu (*symbolic AI*) và phân tích ngữ nghĩa (*semantic parsing*) để chuyển ngôn ngữ của con người thành các dạng mong muốn, chẳng hạn đồ thị tri thức (*knowledge graphs*) hay truy vấn cơ sở dữ liệu. Dù logic trong các hệ thống cổ điển này rất rõ ràng và dễ theo dõi, bản thân các hệ thống lại mong manh. Nếu gặp một từ chúng không biết, hoặc nếu một quy tắc nhỏ bị hỏng, toàn bộ quá trình sẽ sụp đổ.

Khi LLM xuất hiện, bài toán lập luận được phát biểu lại thành một nhiệm vụ sinh văn bản và có thể được giải quyết tốt hơn bằng các kỹ thuật hướng tới lập luận. Sự phát triển của mô hình lập luận trong kỷ nguyên LLM có thể chia thành ba giai đoạn: đoán thẳng đáp án, được prompt để suy nghĩ, và cuối cùng là học cách suy nghĩ.

Các mô hình ngôn ngữ thời kỳ đầu chủ yếu được huấn luyện để dự đoán từ tiếp theo. Khi được hỏi, chúng cố đưa ra đáp án ngay lập tức. Kiểu đoán nhanh này hoạt động tốt với những câu hỏi đơn giản hoặc các nhiệm vụ viết lách. Nhưng với các bài toán toán học hay logic phức tạp, nó không hiệu quả. Các mô hình thường mắc lỗi hoặc bị ảo giác vì chúng không có cách nào để dừng lại và giải bài toán từng bước.

Bước đột phá đầu tiên đến khi các nhà nghiên cứu nhận ra rằng LLM vốn đã sở hữu những năng lực lập luận ẩn giấu: chúng chỉ cần được đánh thức. Chỉ bằng cách dùng prompt CoT [Wei và cộng sự, 2022c] — yêu cầu mô hình "suy nghĩ từng bước" — mô hình buộc phải viết ra quá trình suy nghĩ trước khi đưa ra câu trả lời. Tuy nhiên, phương pháp này vẫn phụ thuộc nhiều vào gợi ý của con người và thực ra không dạy cho mô hình các quy tắc logic nền tảng.

Bước ngoặt lớn hơn gần đây là dùng học tăng cường để huấn luyện mô hình lập luận. Thay vì chỉ dạy mô hình sinh văn bản, các phương pháp học tăng cường huấn luyện những mô hình này thông qua thử và sai (*trial and*

error). Khi mô hình tìm ra đường logic đúng và có được đáp án đúng, nó nhận được phần thưởng. Theo thời gian, các mô hình như o1 của OpenAI [OpenAI, 2024] và DeepSeek-R1 [DeepSeek, 2025] đã cho thấy những năng lực lập luận ấn tượng, chẳng hạn khám phá các ý tưởng khác nhau, tự kiểm tra lại các bước của mình và tự sửa lỗi.

Những năng lực tự sửa lỗi và khám phá này dẫn tới một hướng nghiên cứu quan trọng: mở rộng quy mô lúc suy luận [Snell và cộng sự, 2024]. Ý tưởng rất đơn giản: bằng cách phân bổ nhiều sức mạnh tính toán hơn trong quá trình sinh, mô hình có thể suy nghĩ sâu hơn để giải những bài toán khó hơn. Lý tưởng nhất, một mô hình sẽ điều chỉnh thời gian suy nghĩ dựa trên độ khó thực sự của bài toán. Tuy nhiên, nhiều mô hình lập luận hiện nay áp dụng một cách đơn giản hơn — chúng chỉ áp dụng một quá trình suy nghĩ dài cho mọi truy vấn.

Nhìn rộng hơn, việc mở rộng lượng tính toán lúc suy luận có lợi cho lập luận nói chung. Ngay cả khi một LLM chưa được huấn luyện riêng để lập luận, ta vẫn có thể tăng năng lực giải quyết vấn đề của nó chỉ bằng cách cho nó thêm tính toán để "suy nghĩ". Ví dụ, ta có thể yêu cầu mô hình sinh ra nhiều đường lập luận khác nhau rồi bỏ phiếu chọn đường tốt nhất, hoặc ta có thể dùng các thuật toán tìm kiếm để giúp nó tìm ra các bước logic đúng.

Mô hình lập luận cũng thúc đẩy sự phát triển gần đây của các tác tử dựa trên LLM [Yao và cộng sự, 2023; Wang và cộng sự, 2024b]. Để một tác tử AI tự mình hoàn thành nhiệm vụ, nó cần lập luận để tìm ra cách đạt mục tiêu từng bước. Vì các nhiệm vụ trong thế giới thực hiếm khi diễn ra suôn sẻ hoàn hảo, tác tử còn phải nhận ra khi nào nó mắc lỗi và biết cách sửa. Tất cả những điều này đòi hỏi tác tử có năng lực lập luận tốt.

Sự phát triển nhanh chóng của các mô hình lập luận cũng mở ra những hướng nghiên cứu thú vị nhưng đầy thách thức [Li và cộng sự, 2025; Chen và cộng sự, 2026]. Một hướng tập trung vào hiệu quả. Chẳng hạn, nếu mô hình học được cách điều chỉnh động thời gian suy nghĩ, chúng sẽ không lãng phí tính toán cho những câu hỏi đơn giản mà chỉ "chậm lại" với những câu hỏi khó. Một hướng khác là mở rộng lập luận ra ngoài những lĩnh vực có đáp án đúng-sai rõ ràng như toán học hay lập trình, để xử lý các tình huống mở trong thế giới thực. Ngoài ra, việc giúp mô hình lập luận trên nhiều loại dữ liệu khác nhau, như hình ảnh, âm thanh và môi trường vật lý, là một bước then chốt hướng tới xây dựng các hệ thống AI tiên tiến

hơn.

12.2 Lập luận thông qua prompt

Kỹ thuật prompt là phương pháp đơn giản nhất để khơi gợi lập luận từ LLM. Vì nó không đòi hỏi bất kỳ thay đổi nào về kiến trúc hay tham số của mô hình, ta chỉ cần dùng những văn bản đầu vào được soạn khéo léo để kích thích LLM đưa ra quá trình lập luận. Một trong những phương pháp prompt được dùng rộng rãi nhất là prompt chuỗi suy nghĩ (*chain-of-thought*) (CoT). Ý tưởng cốt lõi là khuyến khích mô hình sinh ra các bước suy diễn logic trung gian trước khi đưa ra đáp án cuối cùng. Điều này thường đạt được bằng cách cung cấp một vài ví dụ giải từng bước trong prompt, hoặc đơn giản là thêm một chỉ dẫn trực tiếp như "Hãy suy nghĩ từng bước" vào sau truy vấn. Dưới đây là một ví dụ về prompt CoT, trong đó chỉ dẫn CoT được tô sáng.

H: Tom mua 3 đôi giày, mỗi đôi \$40, và 2 chiếc áo sơ mi, mỗi chiếc \$20. Nếu anh ấy dùng phiếu giảm giá 10% cho toàn bộ đơn hàng, anh ấy phải trả tổng cộng bao nhiêu?
Đ: Hãy suy nghĩ từng bước.
Trước tiên, tính tổng tiền giày: 3 đôi nhân \$40 bằng \$120. Tiếp theo, tính tổng tiền áo: 2 chiếc nhân \$20 bằng \$40. Tổng tiền trước giảm giá là $\$120 + \$40 = \$160$. Khoản giảm giá là 10% của \$160, tức là \$16. Cuối cùng, trừ khoản giảm giá khỏi tổng tiền: $\$160 - \$16 = \$144$. Vậy đáp án là \$144.

Trong ví dụ này, LLM tạo ra một đường lập luận (*reasoning path*) (phần văn bản gạch dưới) chứa cả quá trình lập luận lẫn đáp án cuối cùng. Một cách hình thức hơn, cho một prompt CoT $\mathbf{x}$[1], LLM sinh ra đầu ra có khả năng cao nhất bằng cách cực đại hóa điểm mô hình hóa ngôn ngữ, như sau

$$\hat{\mathbf{y}} = \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}). \tag{12.1}$$

Ở đây $\Pr(\mathbf{y}|\mathbf{x})$ là xác suất sinh văn bản như trong các mô hình ngôn ngữ tiêu chuẩn. Mỗi đầu ra là một chuỗi token $\mathbf{y} = y_0 ... y_n$. Nó biểu diễn một

[1] Ở đây một prompt CoT gồm cả đầu vào của người dùng lẫn chỉ dẫn CoT.

đường lập luận gồm một chuỗi các bước lập luận:

$$\mathbf{y} = \{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_p}\}. \tag{12.2}$$

Ở đây n_p là số bước lập luận. Mỗi bước $\bar{\mathbf{y}}_i$ là một chuỗi con của $\mathbf{y}$, biểu diễn một bước suy diễn logic hoặc một phép tính trung gian. Lưu ý rằng có nhiều chiến lược để chia một đường lập luận thành các bước rời rạc, chẳng hạn tách theo từng phép toán. Để đơn giản, trong ví dụ này ta coi mỗi câu là một bước lập luận riêng. Do đó ta có

$\bar{\mathbf{y}}_1$: Trước tiên, tính tổng tiền giày: 3 đôi nhân \$40 bằng \$120.
$\bar{\mathbf{y}}_2$: Tiếp theo, tính tổng tiền áo: 2 chiếc nhân \$20 bằng \$40.
$\bar{\mathbf{y}}_3$: Tổng tiền trước giảm giá là $\$120 + \$40 = \$160$.
$\bar{\mathbf{y}}_4$: Khoản giảm giá là 10% của \$160, tức là \$16.
$\bar{\mathbf{y}}_5$: Cuối cùng, trừ khoản giảm giá khỏi tổng tiền: $\$160 - \$16 = \$144$.
$\bar{\mathbf{y}}_6$: Vậy đáp án là \$144.

Chú giải của người dịch: Đọc Phương trình (12.1) và (12.2)

$\Pr(\mathbf{y}|\mathbf{x})$ là xác suất mà mô hình gán cho việc "viết ra văn bản $\mathbf{y}$ khi đã đọc prompt $\mathbf{x}$". Ký hiệu $\arg\max_{\mathbf{y}}$ nghĩa là "chọn ra $\mathbf{y}$ làm cho đại lượng phía sau lớn nhất" (khác với max, vốn cho *giá trị* lớn nhất). Ví dụ, nếu chỉ có ba đầu ra khả dĩ với xác suất 0.2, 0.5, 0.3 thì max là 0.5, còn arg max là chính đầu ra thứ hai. Phương trình (12.2) chỉ nói rằng chuỗi token dài $\mathbf{y}$ có thể được cắt thành n_p khúc liền nhau, mỗi khúc là một "dòng" của lời giải; ở ví dụ trên $n_p = 6$.

Với một đường lập luận như vậy, ta có thể trích xuất đáp án từ bước cuối cùng của nó. Có nhiều cách để cải thiện cách tiếp cận lập luận đơn giản dựa trên CoT này. Chẳng hạn, các kỹ thuật như tự nhất quán (*self-consistency*) lấy mẫu nhiều đường lập luận rồi bỏ phiếu theo đa số để xác định đáp án đáng tin cậy nhất [Wang và cộng sự, 2023c]. Các chiến lược khác bao gồm phân rã bài toán (*problem decomposition*) để chia các truy vấn phức tạp thành những nhiệm vụ con dễ xử lý [Zhou và cộng sự, 2023b], các phương pháp tự phản tư (*self-reflection*) prompt mô hình tự phê bình và sửa các lỗi logic của chính nó [Shinn và cộng sự, 2023; Madaan và cộng sự, 2024], và các khung lập luận cân nhắc (*deliberate reasoning*) khám phá hiệu quả nhiều đường lập luận rẽ nhánh [Yao và

cộng sự, 2024].

Vì nhiều kỹ thuật prompt dựa trên CoT đã được trình bày trước đây, chúng tôi sẽ không đi vào chi tiết ở đây. Bạn đọc được khuyến khích xem lại các kỹ thuật đã đề cập trong Chương 9.

12.3 Lập luận lúc suy luận

Lập luận lúc suy luận (*inference-time reasoning*) liên quan chặt chẽ đến việc mở rộng quy mô trong quá trình suy luận, vốn đã được giới thiệu ở Chương 11. Trong mục này, chúng ta mở rộng thảo luận trước đó và tập trung hơn vào việc cải thiện hiệu năng lập luận của LLM.

12.3.1 Mở rộng quy mô tính toán lúc suy luận

Thành công của LLM phụ thuộc rất nhiều vào việc tăng quy mô sức mạnh tính toán. Một cách phổ biến để cải thiện các mô hình này là mở rộng quy mô lúc huấn luyện (*training-time scaling*). Chỉ đơn giản bằng cách dùng nhiều dữ liệu huấn luyện hơn, kích thước mô hình lớn hơn và nhiều tính toán hơn, các nhà nghiên cứu đã xây dựng được những LLM rất mạnh [Kaplan và cộng sự, 2020; Hoffmann và cộng sự, 2022].

Tuy nhiên, chỉ mở rộng quy mô quá trình huấn luyện là chưa đủ cho các nhiệm vụ lập luận phức tạp. Không giống việc sinh văn bản đơn giản, lập luận cần logic và lập kế hoạch từng bước. Không thể cho mô hình xem mọi đường lập luận khả dĩ trong quá trình huấn luyện. Nếu chỉ dựa vào huấn luyện để giải các bài toán toán học hay logic khó, ta sẽ cần một lượng dữ liệu rất lớn. Lý tưởng nhất, ta muốn mô hình dùng những bước logic đơn giản để tự xây dựng đường lập luận của riêng nó ngay khi đang giải. Vì vậy, việc kích hoạt các năng lực lập luận như thế trong lúc suy luận hấp dẫn hơn.

Vì lý do này, một ý tưởng tự nhiên là mở rộng quy mô tính toán trong giai đoạn suy luận. Nhờ vậy, LLM có thể dành nhiều thời gian hơn để khám phá các bước lập luận khác nhau trước khi đưa ra đáp án cuối cùng. Lợi ích rõ ràng của cách tiếp cận này là nó cho phép sử dụng tài nguyên tính toán linh hoạt và hiệu quả hơn nhiều. Chẳng hạn, với những bài toán toán học hay logic phức tạp, LLM có thể phân bổ nhiều thời gian và sức

Bảng 12.1: So sánh mở rộng quy mô lúc huấn luyện và lúc suy luận

Đặc điểm	Mở rộng quy mô lúc huấn luyện	Mở rộng quy mô lúc suy luận
Cách hoạt động	Làm cho mô hình lớn hơn và nạp cho nó nhiều dữ liệu hơn trong giai đoạn học ban đầu.	Giữ nguyên kích thước mô hình, nhưng cho nó thêm tính toán để "suy nghĩ" (ví dụ: khám phá các đường khác nhau) trước khi trả lời.
Chi phí huấn luyện	Cực kỳ cao. Rất tốn kém và khó sửa nếu có gì đó sai.	Thấp đến trung bình. Có thể phải tốn chi phí huấn luyện thêm các mô hình nhỏ hơn để kiểm tra câu trả lời của nó.
Chi phí sử dụng	Dự đoán được. Chi phí sinh một token chủ yếu phụ thuộc vào kích thước mô hình và độ dài ngữ cảnh.	Linh hoạt nhưng có thể trở nên rất cao. Mô hình "suy nghĩ" càng sâu thì càng tốn kém.
Yêu cầu phần cứng (suy luận)	Cần băng thông bộ nhớ (*memory bandwidth*) cao để đọc và ghi dữ liệu nhanh.	Cần băng thông cao như vậy, cộng thêm dung lượng bộ nhớ lớn để khám phá nhiều đường lập luận dài.
Dữ liệu cần thiết	Lượng khổng lồ văn bản tổng quát.	Các ví dụ chất lượng cao, giải từng bước (như chứng minh toán học) và một cách rõ ràng để chấm xem một đáp án đúng hay sai.

mạnh hơn, và nhờ đó "suy nghĩ" lâu hơn. Trong nhiều trường hợp, một mô hình nhỏ hơn được cho thêm thời gian lập luận lúc suy luận thậm chí có thể vượt trội một mô hình lớn hơn nhiều nhưng bị buộc phải trả lời ngay lập tức. Chúng tôi tóm tắt sự khác biệt giữa mở rộng quy mô lúc huấn luyện và mở rộng quy mô lúc suy luận trong Bảng 12.1.

Ta có thể xem mở rộng quy mô lúc suy luận như một bài toán **mở rộng quy mô tìm kiếm** (*search scaling*). Khi cho mô hình thêm tính toán để "suy nghĩ", về bản chất ta đang cho phép nó khám phá một không gian các bước logic khả dĩ rộng hơn, và có cơ hội cao hơn nhiều để tìm ra lời giải tối ưu. Điều này dẫn đến ba câu hỏi then chốt:

- Làm thế nào để sinh ra các đường lập luận khác nhau?
- Làm thế nào để tìm kiếm qua các bước khả dĩ để tìm ra bước tốt nhất?
- Làm thế nào để kiểm tra xem các bước trung gian và đáp án cuối cùng có đúng không?

Trong các tiểu mục tiếp theo, chúng ta sẽ giải quyết những câu hỏi này bằng cách thảo luận ba chủ đề: lấy mẫu nhiều đường lập luận (Mục 12.3.2), các chiến lược tìm kiếm cho lập luận (Mục 12.3.3) và kiểm chứng (Mục 12.3.4).

12.3.2 Lấy mẫu nhiều đường lập luận

Cách trực tiếp nhất để mở rộng quy mô tính toán lúc suy luận là sinh ra không chỉ một mà nhiều đường lập luận cho một đầu vào duy nhất. Bằng cách sinh ra nhiều đường khác nhau, ta cho LLM nhiều cơ hội hơn để tìm ra logic đúng và đáp án tương ứng.

Để làm được điều này, thông lệ phổ biến là đưa tính ngẫu nhiên vào quá trình sinh văn bản. Một phương pháp điển hình là dùng lấy mẫu với nhiệt độ (*temperature sampling*), kết hợp với lấy mẫu top-k hoặc lấy mẫu hạt nhân (*nucleus sampling*) (top-p). Trong sinh tự hồi quy (*auto-regressive generation*) tiêu chuẩn, tại mỗi vị trí i, LLM đưa ra một phân phối xác suất $\Pr(\cdot|\mathbf{x}, \mathbf{y}_{<i})$ trên bộ từ vựng V cho token tiếp theo y_i dựa trên các logit (*logits*) u_{y_i} của nó. Lấy mẫu với nhiệt độ thay đổi phân phối này bằng cách chia các logit cho một tham số nhiệt độ $T > 0$ trước khi áp

dụng hàm Softmax:

$$\Pr(y_i|\mathbf{x}, \mathbf{y}_{<i}) = \frac{\exp(u_{y_i}/T)}{\sum_{y'_i \in V} \exp(u_{y'_i}/T)}. \tag{12.3}$$

Nhiệt độ cao hơn làm phân phối "phẳng" hơn, nhờ đó khuyến khích mô hình khám phá nhiều kiểu bước lập luận đa dạng hơn. Tuy nhiên, tính ngẫu nhiên không bị ràng buộc có thể dẫn đến văn bản suy biến hoặc phi logic do thỉnh thoảng lấy mẫu từ "đuôi dài" gồm các token có xác suất thấp. Để duy trì chất lượng của các đường được sinh ra, lấy mẫu với nhiệt độ thường được kết hợp với các phương pháp cắt bớt bộ từ vựng:

- **Lấy mẫu top-k** giới hạn tập lấy mẫu chỉ còn k token tiếp theo có khả năng cao nhất, rồi phân phối lại khối lượng xác suất giữa chúng.
- **Lấy mẫu hạt nhân (top-p)** dùng chiến lược cắt bớt động. Nó chọn tập nhỏ nhất gồm các token xếp hạng cao nhất có xác suất tích lũy vượt quá một ngưỡng p.

Chú giải của người dịch: Nhiệt độ, top-k và top-p bằng số cụ thể

Giả sử chỉ có 3 token với logit $u = (2, 1, 0)$. Với $T = 1$: $e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1$, tổng ≈ 11.11, nên xác suất $\approx (0.67, 0.24, 0.09)$. Với $T = 2$ ta chia logit cho 2 được $(1, 0.5, 0)$: $e^1 \approx 2.72$, $e^{0.5} \approx 1.65$, 1, tổng ≈ 5.37, nên xác suất $\approx (0.51, 0.31, 0.19)$ — "phẳng" hơn, các token ít khả năng được chọn thường hơn. Ngược lại T nhỏ (gần 0) làm phân phối dồn gần hết vào token đứng đầu. Với phân phối $(0.67, 0.24, 0.09)$: top-k với $k = 2$ chỉ giữ hai token đầu và chia lại thành $(0.67, 0.24)/0.91 \approx (0.74, 0.26)$; top-$p$ với $p = 0.8$ cũng giữ hai token đầu vì $0.67 < 0.8$ nhưng $0.67 + 0.24 = 0.91 > 0.8$.

Bằng cách áp dụng các chiến lược giải mã ngẫu nhiên này, cho một đầu vào $\mathbf{x}$, LLM có thể lấy mẫu một tập các chuỗi đầu ra đa dạng (tức là các đường lập luận ứng viên):

$$\mathcal{Y}^{\text{top}} = \{\hat{\mathbf{y}}^1, \ldots, \hat{\mathbf{y}}^N\}. \tag{12.4}$$

Ở đây N biểu diễn ngân sách tính toán (*compute budget*) lúc suy luận. N càng lớn thì ta càng tốn nhiều tính toán để khám phá các đường lập luận khác nhau. Khi đã có tập đường đa dạng này, bài toán tiếp theo là chọn ra đường tốt nhất. Có hai phương pháp được dùng rộng rãi cho việc này: Best-of-N và tự nhất quán.

1. Best-of-N (BoN)

Trước đây chúng ta đã giới thiệu lấy mẫu Best-of-N ở Chương 10 như một phương pháp căn chỉnh lúc suy luận (*inference-time alignment*). Trong bối cảnh đó, ta dùng một mô hình phần thưởng $r(\mathbf{x}, \hat{\mathbf{y}})$ để chọn đầu ra phù hợp nhất với sở thích của con người.

Với các nhiệm vụ lập luận, khái niệm lấy mẫu BoN vẫn giữ nguyên, nhưng mục tiêu thay đổi. Thay vì một mô hình phần thưởng theo sở thích, "giám khảo" thường là một bộ kiểm chứng (*verifier*) (mà ta có thể ký hiệu là một hàm chấm điểm tổng quát v) được thiết kế để đánh giá mức độ chặt chẽ về logic của một đường lập luận. Bộ kiểm chứng gán một điểm số $v(\mathbf{x}, \hat{\mathbf{y}})$ cho mỗi đường được lấy mẫu, và ta chọn đường lập luận tốt nhất $\hat{\mathbf{y}}_{\text{best}}$ làm cực đại điểm số đó

$$\hat{\mathbf{y}}_{\text{best}} = \underset{\hat{\mathbf{y}} \in \{\hat{\mathbf{y}}^1, \ldots, \hat{\mathbf{y}}^N\}}{\arg\max} \; v(\mathbf{x}, \hat{\mathbf{y}}). \tag{12.5}$$

Đáp án cuối cùng sau đó được trích xuất từ đường có điểm cao nhất này. Để hiểu quá trình BoN, hãy xét một phép so sánh trực quan. Hãy tưởng tượng có một lớp học gồm N học sinh đang giải một bài toán. Một giáo viên (bộ kiểm chứng) chấm tất cả giấy nháp của các em, rồi chọn ra học sinh có điểm cao nhất.

Dù lấy mẫu BoN rất mạnh, hiệu quả của nó phụ thuộc vào chất lượng của bộ kiểm chứng. Nếu bộ kiểm chứng yếu, nó có thể dễ dàng bị đánh lừa bởi những lập luận sai mà tình cờ đi đến đáp án cuối cùng đúng (tức là khai thác lỗ hổng phần thưởng (*reward hacking*)). Việc thiết kế bộ kiểm chứng sẽ được bàn chi tiết hơn ở Mục 12.3.4.

2. Tự nhất quán

Một cách tiếp cận thay thế cho việc chọn đường tốt nhất bằng một bộ kiểm chứng được học là tự nhất quán. Tự nhất quán dựa trên trực giác rằng trong các nhiệm vụ lập luận phức tạp, thường có nhiều đường logic khác nhau cùng dẫn đến một đáp án đúng, trong khi các đường lập luận sai có xu hướng đa dạng và dẫn đến những lỗi ngẫu nhiên. Thay vì chấm điểm quá trình lập luận, tự nhất quán trực tiếp lấy biên (*marginalize out*) các đường lập luận bằng cách bỏ phiếu theo đa số trên các đáp án cuối cùng. Một cách hình thức, gọi A là tập tất cả các đáp án cuối cùng khả dĩ. Với mỗi đường được lấy mẫu $\hat{\mathbf{y}}^k$, ta trích xuất đáp án dự đoán cuối cùng

của nó $\hat{a}^k = \text{Extract}(\hat{\mathbf{y}}^k)$. Chiến lược tự nhất quán chọn đáp án $\hat{a}_{\text{sc}}$ xuất hiện nhiều nhất trong N đường được lấy mẫu:

$$\hat{a}_{\text{sc}} = \arg\max_{a \in A} \sum_{k=1}^{N} \mathbf{1}(\text{Extract}(\hat{\mathbf{y}}^k) = a), \tag{12.6}$$

trong đó $\mathbf{1}(\cdot)$ là hàm chỉ thị (*indicator function*).

Chú giải của người dịch: BoN và tự nhất quán qua một ví dụ

Giả sử $N = 5$ đường lập luận cho bài toán của Tom cho các đáp án: \$144, \$160, \$144, \$144, \$136. Hàm chỉ thị $\mathbf{1}$(mệnh đề) bằng 1 nếu mệnh đề đúng và bằng 0 nếu sai; nên tổng trong Phương trình (12.6) chính là *số phiếu* của mỗi đáp án: \$144 được 3 phiếu, \$160 và \$136 mỗi đáp án 1 phiếu. Tự nhất quán chọn \$144. Còn BoN không đếm phiếu mà nhờ bộ kiểm chứng chấm điểm cả 5 lời giải (chẳng hạn $0.9, 0.4, 0.7, 0.8, 0.3$) và lấy lời giải có điểm cao nhất (lời giải thứ nhất).

3. Cân bằng giữa độ đa dạng và chất lượng khi lấy mẫu

Khi lấy mẫu nhiều đường lập luận để mở rộng quy mô tính toán lúc suy luận, ta cần tìm sự đánh đổi giữa độ đa dạng và chất lượng. Trong các nhiệm vụ lập luận, ta thường mong muốn một tập đường lấy mẫu đa dạng, điều này làm tăng khả năng tập đó chứa logic đúng. Một mẹo kinh nghiệm phổ biến là tăng nhiệt độ lấy mẫu. Bằng cách làm phân phối phẳng hơn, quá trình lấy mẫu tránh chỉ tập trung vào một vài đường có xác suất cao. Kết quả là các đường có xác suất thấp hơn có nhiều khả năng được chọn hơn, và mô hình có thể khám phá nhiều kiểu đường lập luận đa dạng hơn trong N mẫu. Tuy nhiên, khi nhiệt độ tăng, dù độ đa dạng của các mẫu được cải thiện, chất lượng logic trung bình của từng đường có thể giảm.

Để đạt được sự cân bằng giữa độ đa dạng và chất lượng, một phương pháp đơn giản là sửa đổi mục tiêu tìm kiếm trong giai đoạn suy luận bằng cách đưa vào một số hạng phạt hoặc thưởng tường minh. Trong thiết lập này, độ đa dạng có thể được mô hình hóa toán học như một hàm được kết hợp với xác suất sinh ban đầu. Một cách hình thức hơn, mục tiêu tìm kiếm đã sửa đổi có thể được viết là

$$\text{score}(\mathbf{x}, \mathbf{y}) = \log \Pr(\mathbf{y}|\mathbf{x}) + \lambda D(\mathbf{y}, \mathcal{Y}^{\text{top}}). \tag{12.7}$$

Ở đây $\log \Pr(\mathbf{y}|\mathbf{x})$ ưu tiên các đầu ra có xác suất cao hơn, còn $D(\mathbf{y}, \mathcal{Y}^{\text{top}})$

đóng vai trò số hạng phạt hoặc thưởng nhằm khuyến khích sự đa dạng so với các đường đã lấy mẫu $\mathcal{Y}^{\text{top}}$. Siêu tham số λ điều khiển mức độ của sự đánh đổi này. Việc thiết kế $D(\cdot)$ có thể khá linh hoạt. Chẳng hạn, một hàm phạt đơn giản có thể giảm điểm của các token hoặc n-gram đã xuất hiện thường xuyên trong $\mathcal{Y}^{\text{top}}$. Ngược lại, một hàm thưởng có thể được thiết kế để khen thưởng những khác biệt rõ rệt về ngữ nghĩa. Nhờ vậy, LLM có thể tạo ra những cấu trúc lập luận mới thay vì chỉ diễn đạt lại chút ít cùng một ý.

Nhìn từ góc độ rộng hơn, thế lưỡng nan khi lấy mẫu này cũng liên quan đến sự đánh đổi giữa khám phá và khai thác (*exploration–exploitation trade-off*) được nghiên cứu trong học tăng cường [Sutton và Barto, 2018]. Trong tinh chỉnh dựa trên RL, một LLM có thể được xem là một chính sách π_θ sinh ra đường lập luận $\mathbf{y}$ cho một câu hỏi $\mathbf{x}$. Một mục tiêu có chính quy hóa tổng quát có thể được viết là

$$\max_{\pi_\theta} \; \mathbb{E}_{\mathbf{y}\sim\pi_\theta}\left[R(\mathbf{x},\mathbf{y})\right] - \beta D_{\text{KL}}\left(\pi_\theta(\cdot|\mathbf{x}) \,\|\, \pi_{\text{ref}}(\cdot|\mathbf{x})\right) + \alpha\,\text{Entropy}\left(\pi_\theta(\cdot|\mathbf{x})\right), \tag{12.8}$$

trong đó $R(\mathbf{x},\mathbf{y})$ đo chất lượng của đường lập luận được sinh ra. Số hạng độ phân kỳ KL (*KL-divergence*) ràng buộc chính sách được học phải luôn gần với chính sách tham chiếu (*reference policy*). Mục đích chính của nó là ngăn chính sách bị trôi dạt quá mức, và nhờ đó giảm các hành vi không mong muốn như khai thác lỗ hổng phần thưởng hay suy giảm chất lượng ngôn ngữ. Ngược lại, số hạng entropy khuyến khích tường minh chính sách duy trì một phân phối ít tập trung hơn, và vì thế thúc đẩy việc khám phá trong quá trình huấn luyện.

Chú giải của người dịch: Đọc Phương trình (12.8) theo từng số hạng

Có ba "lực" kéo chính sách π_θ (tức là LLM với tham số θ): (1) $\mathbb{E}_{\mathbf{y}\sim\pi_\theta}[R]$ là *phần thưởng trung bình* khi để mô hình tự sinh lời giải — càng lớn càng tốt; (2) D_{KL} đo mức "khác biệt" giữa phân phối của mô hình mới và mô hình tham chiếu ban đầu (bằng 0 khi hai phân phối trùng nhau); dấu trừ và hệ số β phạt việc đi quá xa mô hình gốc, giống như sợi dây níu; (3) Entropy đo độ "trải rộng" của phân phối: một phân phối dồn hết vào một lựa chọn có entropy bằng 0, còn phân phối đều có entropy lớn nhất; dấu cộng và hệ số α thưởng cho việc giữ nhiều lựa chọn, tức là khuyến khích khám phá. Các hệ số $\beta, \alpha \geq 0$ quyết định lực nào mạnh hơn.

Có một mối liên hệ về mặt khái niệm giữa mục tiêu RL và sự đánh đổi giữa đa dạng và chất lượng lúc suy luận, dù hai mục tiêu hoạt động trên những không gian khác nhau. Lúc suy luận, thủ tục tìm kiếm điều khiển mức độ rộng mà nó khám phá các đường lập luận do một mô hình cố định sinh ra. Trong tinh chỉnh dựa trên RL, chính quy hóa KL điều khiển việc chính sách được học được phép đi xa chính sách tham chiếu đến mức nào. Hệ số KL lớn hơn giữ chính sách gần mô hình tham chiếu, trong khi hệ số nhỏ hơn cho phép quá trình tối ưu hóa khám phá một vùng rộng hơn của không gian chính sách để tìm những hành vi có phần thưởng cao hơn. Những thay đổi chính sách như vậy có thể giúp mô hình sinh ra các đường lập luận ít có khả năng xuất hiện dưới chính sách tham chiếu, và vì thế có thể dẫn đến một dải đầu ra rộng hơn.

Cả tìm kiếm lúc suy luận lẫn RL có chính quy hóa KL đều đòi hỏi một sự cân bằng giữa hành vi thận trọng và khám phá rộng hơn. Điểm khác biệt là các phương pháp lúc suy luận khám phá các đầu ra khác nhau dưới một mô hình cố định, còn RL tìm kiếm trong không gian chính sách bằng cách cập nhật tham số mô hình. Trong cả hai trường hợp, cho phép khám phá rộng hơn có thể phát hiện ra những hành vi lập luận tốt hơn và đa dạng hơn, nhưng khám phá quá mức có thể làm giảm chất lượng hoặc độ ổn định.

12.3.3 Các chiến lược tìm kiếm cho lập luận

Trong thực tế, việc tìm đường lập luận tối ưu có thể được phát biểu trực tiếp thành một bài toán tìm kiếm kinh điển: ta bắt đầu từ một trạng thái ban đầu, thực hiện một hành động để sinh ra một bước lập luận, mở rộng trạng thái hiện tại thành một tập trạng thái mới, và lặp lại quá trình này cho đến khi một điều kiện dừng được thỏa mãn. Vì chúng ta đã giới thiệu một số thuật toán tìm kiếm kinh điển (như tìm kiếm cây và tìm kiếm chùm) ở Chương 5 và Chương 11, ta sẽ không nhắc lại chi tiết các thuật toán này ở đây. Thay vào đó, ta tập trung vào cách ánh xạ việc tìm kiếm đường lập luận vào các khung tìm kiếm kinh điển này.

Để áp dụng một thuật toán tìm kiếm vào lập luận của LLM, ta cần định nghĩa các thành phần tiêu chuẩn của một bài toán tìm kiếm:

- **Trạng thái tìm kiếm.** Với lập luận của LLM, trạng thái là ngữ cảnh mà mô hình đã xây dựng được cho đến lúc đó. Cụ thể, tại bước t,

trạng thái s_t gồm truy vấn gốc $\mathbf{x}$ kết hợp với đường lập luận dở dang đã sinh ra đến thời điểm này: $s_t = (\mathbf{x}, \bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_t)$.

- **Hành động.** Một hành động là việc sinh ra bước lập luận tiếp theo (một đoạn văn bản) $\bar{\mathbf{y}}_{t+1}$.
- **Mô hình chuyển trạng thái.** Thực hiện một hành động đơn giản là nối đoạn văn bản mới vào trạng thái hiện có. Điều này tạo ra trạng thái tiếp theo: $s_{t+1} = (s_t, \bar{\mathbf{y}}_{t+1}) = (\mathbf{x}, \bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_t, \bar{\mathbf{y}}_{t+1})$.
- **Hàm đánh giá trạng thái.** Đây là một hàm chấm điểm đánh giá mức độ đúng đắn về logic của trạng thái hiện tại trên đường đến đáp án cuối cùng. Ví dụ, để đánh giá các trạng thái tìm kiếm, ta có thể dùng các heuristic (*heuristics*) được lập trình cứng, hoặc một bộ kiểm chứng nơ-ron đã được huấn luyện.
- **Điều kiện dừng.** Tiêu chí để dừng tìm kiếm. Điều này xảy ra khi mô hình sinh ra tường minh một chuỗi đáp án cuối cùng, hoặc khi quá trình tìm kiếm chạm tới một ngân sách tính toán định trước (ví dụ: độ sâu tối đa của các nút được khám phá).

Chú giải của người dịch: Tìm kiếm đường lập luận giống như đi trong mê cung

Hãy hình dung mỗi trạng thái là "đoạn lời giải đã viết đến giờ". Từ một trạng thái, mô hình có thể viết tiếp nhiều câu khác nhau (nhiều hành động), mỗi câu dẫn đến một trạng thái mới — giống như đứng ở ngã rẽ trong mê cung. Nếu mỗi bước có 3 lựa chọn và lời giải dài 4 bước thì có $3^4 = 81$ đường khả dĩ; hàm đánh giá trạng thái giống như một người dẫn đường cho biết ngã rẽ nào "có vẻ" gần lối ra, giúp ta không phải thử hết tất cả các đường.

Khi nhiệm vụ lập luận đã được phát biểu thành một bài toán tìm kiếm, có thể áp dụng nhiều thuật toán tìm kiếm khác nhau để khám phá không gian lập luận một cách hiệu quả. Chẳng hạn, các phương pháp tìm kiếm cây đơn giản, như tìm kiếm theo chiều rộng (*breadth-first search*) và tìm kiếm theo chiều sâu (*depth-first search*), có thể được dùng để mô hình hóa quá trình lập luận của LLM. Trong các phương pháp này, mô hình khám phá nhiều ý nghĩ ứng viên ở mỗi bước và đánh giá chúng để quyết định nên đi tiếp theo một đường hay quay lui (*backtrack*) khi gặp ngõ cụt. Với các nhiệm vụ có độ bất định cao hơn hoặc phần thưởng đến muộn, tìm

kiếm cây Monte Carlo (*Monte Carlo tree search*) thường được sử dụng. Phương pháp này giúp mô hình cân bằng giữa khám phá các đường lập luận mới và khai thác các bước đã biết. Đôi khi nó còn tích hợp phản hồi từ môi trường bên ngoài và cơ chế tự phản tư để tinh chỉnh quá trình tìm kiếm [Yao và cộng sự, 2024; Hao và cộng sự, 2023; Zhou và cộng sự, 2024].

Để giải quyết sự dư thừa của cấu trúc cây, khi nhiều đường lập luận khác nhau có thể hội tụ về cùng một kết luận trung gian, một số phương pháp mô hình hóa không gian lập luận như một đồ thị (chẳng hạn đồ thị có hướng không chu trình (*directed acyclic graph*), DAG) [Zhang và cộng sự, 2025]. Bằng cách này, mô hình có thể xây dựng các cấu trúc không gian tìm kiếm hiệu quả hơn, và nắm bắt được các phụ thuộc lẫn nhau phi tuyến của suy diễn logic. Ngoài các thuật toán tìm kiếm bên ngoài, cũng có thể nội tại hóa động lực của quá trình tìm kiếm trực tiếp vào tham số của LLM. Ví dụ, bằng cách huấn luyện LLM trên các vết thực thi của những thuật toán tìm kiếm heuristic như A*, chúng có thể học cách sinh ra một cách ngầm định quá trình tìm kiếm và các kế hoạch tối ưu mà không cần dựa vào một kịch bản điều khiển bên ngoài [Lehnert và cộng sự, 2025].

Dù mở rộng quy mô tính toán lúc suy luận thông qua tìm kiếm là một chiến lược mạnh, mối quan hệ giữa công sức tìm kiếm và hiệu năng lập luận không đơn giản là tuyến tính. Theo trực giác, ta có thể kỳ vọng rằng cho mô hình nhiều thời gian khám phá hơn sẽ luôn mang lại câu trả lời tốt hơn. Tuy nhiên, trong thực tế, mức tăng hiệu năng nhanh chóng chững lại. Một lý do chính là giới hạn năng lực vốn có của LLM: nếu mô hình thiếu tri thức để giải một bài toán, dù tìm kiếm bao nhiêu cũng không thể tạo ra logic đúng từ hư không. Ngoài ra, khi công sức tìm kiếm tăng lên, độ đa dạng của các đường được sinh ra giảm xuống, nghĩa là mô hình lãng phí tính toán vào việc khám phá những ý tưởng trùng lặp, ít mang lại giá trị mới.

12.3.4 Kiểm chứng

Tiếp theo, chúng ta thảo luận bài toán kiểm chứng trong lập luận. Lưu ý rằng dù trong mục này ta tập trung vào các phương pháp lúc suy luận, nhiều bộ kiểm chứng dùng trong mô hình lập luận vẫn cần được huấn luyện. Vì vậy, chúng tôi cũng trình bày một số chi tiết về việc huấn luyện

các mô hình này trong phần thảo luận dưới đây.

1. Mô hình phần thưởng theo quá trình

Có hai mô thức chính để kiểm chứng kết quả lập luận: mô hình phần thưởng theo kết quả (*outcome reward models*) (ORM) và mô hình phần thưởng theo quá trình (*process reward models*) (PRM) [Uesato và cộng sự, 2022; Lightman và cộng sự, 2024]. Một ORM chỉ đánh giá đáp án cuối cùng của một đường lập luận. Dù dễ huấn luyện hơn, ORM thường không đủ cho các nhiệm vụ lập luận vì chúng gặp vấn đề "dương tính giả (*false positives*)". Để minh họa, xét một bài toán đơn giản:

> Tìm x: $2x + 4 = 10$.

Một mô hình có thể sinh ra đường lập luận sau:

> $2x + 4 = 10$ nghĩa là x phải bằng một nửa của 10, tức là 5.
> Nhưng $5 - 2 = 3$.
> Do đó, $x = 3$.

Mặc dù đáp án cuối cùng (tức là 3) đúng, các bước logic hoàn toàn sai. Nếu chỉ dùng ORM, đường này có thể nhận điểm cao và vô tình củng cố logic sai. Để giải quyết vấn đề này, ta có thể dùng PRM để đánh giá tính đúng đắn của từng bước riêng lẻ trong đường lập luận. Cho một đường lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_{n_p}\}$, một PRM gán một điểm đúng đắn r_t cho bước hiện tại $\bar{\mathbf{y}}_t$, với điều kiện là đầu vào $\mathbf{x}$ và tất cả các bước trước đó $\bar{\mathbf{y}}_{<t}$. Ta có thể định nghĩa hàm kiểm chứng ở mức bước là

$$r_t \;=\; v_{\text{prm}}(\bar{\mathbf{y}}_t|\mathbf{x}, \bar{\mathbf{y}}_{<t}). \tag{12.9}$$

Ở đây $v_{\text{prm}}(\cdot)$ là một mạng nơ-ron được huấn luyện để dự đoán điểm đúng đắn. Ví dụ, ta có thể gắn một đầu phân loại (*classification head*) đơn giản (chẳng hạn một tầng tuyến tính theo sau là hàm kích hoạt sigmoid) vào biểu diễn đầu ra của một LLM có sẵn.

Để xác định chất lượng tổng thể của toàn bộ đường lập luận, ta có thể tổng hợp các điểm số ở mức bước này. Một cách đơn giản là lấy điểm nhỏ nhất trên tất cả các bước. Nói cách khác, ta chỉ coi lập luận là đúng nếu "mắt xích yếu nhất" của nó chặt chẽ về logic. Khi đó điểm kiểm chứng

được cho bởi

$$v(\mathbf{x}, \mathbf{y}) = \min_{1 \le t \le n_p} v_{\text{prm}}(\bar{\mathbf{y}}_t | \mathbf{x}, \bar{\mathbf{y}}_{<t}). \tag{12.10}$$

Ngoài ra, ta có thể định nghĩa điểm kiểm chứng là trung bình của các điểm ở mức bước

$$v(\mathbf{x}, \mathbf{y}) = \frac{1}{n_p} \sum_{1 \le t \le n_p} v_{\text{prm}}(\bar{\mathbf{y}}_t | \mathbf{x}, \bar{\mathbf{y}}_{<t}). \tag{12.11}$$

Chú giải của người dịch: Lấy min hay lấy trung bình?

Giả sử một lời giải 4 bước được PRM chấm 0.95, 0.9, 0.2, 0.95 (bước 3 rất đáng ngờ). Theo Phương trình (12.10), điểm cả đường là min $= 0.2$: chỉ cần một bước sai là cả lời giải bị đánh giá thấp, giống như một sợi xích chỉ chắc bằng mắt xích yếu nhất. Theo Phương trình (12.11), điểm là $(0.95 + 0.9 + 0.2 + 0.95)/4 = 0.75$: bước sai bị "pha loãng" bởi các bước tốt. Cách lấy min khắt khe hơn, phù hợp khi một lỗi duy nhất đủ làm hỏng kết quả.

Để huấn luyện một mô hình kiểm chứng, ta cần một tập dữ liệu đã được gán nhãn. Khác với huấn luyện ORM, vốn chỉ cần biết đáp án đúng cuối cùng ở cuối đường, huấn luyện PRM đòi hỏi giám sát ở mức bước (*step-level supervision*). Có hai cách thường dùng để xây dựng tập dữ liệu huấn luyện PRM:

- **Gán nhãn bởi con người:** Các chuyên gia tự tay giải bài toán và gán nhãn "đúng" hoặc "sai" cho từng bước trung gian do LLM sinh ra. Dù phương pháp này có thể tạo ra các bộ kiểm chứng chính xác, nó tốn kém và khó mở rộng quy mô, đặc biệt với các nhiệm vụ toán học nâng cao hay lập trình đòi hỏi chuyên gia lĩnh vực.

- **Kiểm chứng tự động thông qua triển khai thử (rollout):** Ta cũng có thể dùng các phương pháp tự động để ước lượng giá trị của một bước [Wang và cộng sự, 2024c]. Chẳng hạn, nếu có một tập dữ liệu với đáp án cuối cùng đã biết (như các đề thi toán), ta có thể dùng **triển khai thử Monte Carlo** (*Monte Carlo rollouts*). Từ một bước trung gian $\bar{\mathbf{y}}_t$, ta prompt LLM tiếp tục sinh ra nhiều đường hoàn chỉnh. Nếu một tỷ lệ cao các đường này cuối cùng đạt đến đáp án đúng chuẩn a^*, ta có thể giả định rằng bước trung gian $\bar{\mathbf{y}}_t$ đang đi đúng hướng. Một cách hình thức, giá trị ước lượng $\hat{r}_t$ của một bước

có thể được định nghĩa là xác suất kỳ vọng đạt được đáp án đúng:

$$\hat{r}_t \;=\; \mathbb{E}_{\mathbf{y}_{>t}\sim\pi_\theta}\left[\mathbf{1}(\text{Extract}(\mathbf{y}_{>t}) = a^*)\right]. \tag{12.12}$$

Ở đây $\mathbf{y}_{>t}$ là phần đường còn lại tính từ $\bar{\mathbf{y}}_t$, và $\text{Extract}(\mathbf{y}_{>t})$ biểu diễn đáp án được trích xuất từ đường $\mathbf{y}_{>t}$. Khi các giá trị này đã được ước lượng thông qua triển khai thử, có thể huấn luyện một PRM để dự đoán $\hat{r}_t$.

Chú giải của người dịch: Phương trình (12.12) chỉ là "tỷ lệ về đích đúng"

Kỳ vọng của một hàm chỉ thị chính là xác suất để điều kiện bên trong xảy ra. Trong thực tế ta ước lượng nó bằng tần suất: từ bước $\bar{\mathbf{y}}_t$, cho mô hình viết tiếp, chẳng hạn, 10 lời giải hoàn chỉnh; nếu 7 lời giải ra đúng đáp án a^* thì $\hat{r}_t \approx 7/10 = 0.7$. Bước nào "dẫn đường tốt" sẽ có tỷ lệ này cao; bước đã sai thì hầu như mọi lời giải viết tiếp đều sai, nên tỷ lệ gần 0. Đây giống cách tính xác suất bằng thống kê tần suất ở THPT.

2. Bộ kiểm chứng sinh

Tất cả các bộ kiểm chứng nói trên đều đòi hỏi sửa đổi kiến trúc và công sức huấn luyện thêm, ví dụ ta cần huấn luyện một đầu phân loại để đưa ra một điểm số. Với năng lực sinh văn bản mạnh của LLM, một cách tiếp cận tự nhiên khác là trực tiếp prompt LLM đánh giá các bước lập luận và sinh ra phản hồi kiểm chứng bằng ngôn ngữ tự nhiên. Cách tiếp cận này được gọi là **kiểm chứng sinh** (*generative verification*) [Zheng và cộng sự, 2023].

Thay vì sửa đổi kiến trúc mô hình để đưa ra một giá trị vô hướng duy nhất, ta phát biểu nhiệm vụ kiểm chứng như một bài toán sinh văn bản tiêu chuẩn. Bằng cách đưa truy vấn và các bước trung gian trở lại vào LLM, ta có thể yêu cầu nó đóng vai giám khảo. Điều này có nghĩa là mô hình hành động như một giáo viên viết lời phê bên lề bài kiểm tra, thay vì chỉ cho một điểm số cuối cùng.

Để hiểu cách hoạt động, hãy xem một ví dụ đơn giản. Giả sử mô hình đang giải một hệ phương trình:

Bài toán: Tìm x và y biết $2x + y = 10$ và $x - y = 2$.

Ta giả sử mô hình đã sinh ra hai bước đúng:

> *Từ phương trình thứ hai, ta có thể biểu diễn x theo y: $x = y + 2$.*
> *Thế x vào phương trình thứ nhất: $2(y + 2) + y = 10$.*

Bây giờ nó đang thử bước thứ ba, trong đó có một lỗi tính toán:

> *Khai triển phương trình được $2y + 2 + y = 10$, rút gọn thành $3y + 2 = 10$.*

Ta có thể kết hợp truy vấn gốc, bước 1 và 2, cùng bước hiện tại 3 với một prompt kiểm chứng (ví dụ: "*Dựa trên các bước trước, hãy kiểm tra xem logic ở bước hiện tại có đúng không và giải thích kết luận của bạn.*"). Bộ kiểm chứng sinh khi đó có thể đưa ra một lời phê bình bằng văn bản như sau:

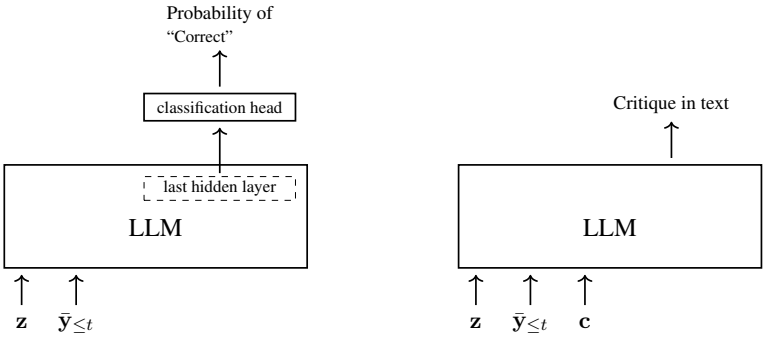
> *Nhận xét: Sai. Ở bước hiện tại, khi phân phối số 2 vào $(y+2)$ từ Bước 2, bạn phải nhân cả hai số hạng trong ngoặc với 2. Kết quả phải là $2y + 4$, không phải $2y+2$. Khai triển đúng là $2y+4+y = 10$, rút gọn thành $3y+4 = 10$.*

Để hình thức hóa quá trình kiểm chứng sinh này, ta giả sử đầu vào gốc $\mathbf{x}$ gồm một truy vấn $\mathbf{z}$ và một chỉ dẫn $\mathbf{c}$ yêu cầu mô hình kiểm chứng bước dựa trên ngữ cảnh. Ta có thể nối truy vấn $\mathbf{z}$, lịch sử lập luận $\bar{\mathbf{y}}_{\leq t}$ và chỉ dẫn $\mathbf{c}$ thành một văn bản dài. Sau đó, ta đưa văn bản này làm đầu vào cho một LLM và thu được một lời phê bình bằng ngôn ngữ tự nhiên $\mathbf{p}_t$:

$$\mathbf{p}_t \sim \Pr(\cdot|\mathbf{z}, \bar{\mathbf{y}}_{\leq t}, \mathbf{c}). \tag{12.13}$$

Ở đây $\Pr(\cdot)$ biểu diễn phân phối xác suất do một LLM có sẵn mô hình hóa. Như vậy ta có thể trực tiếp dùng các LLM mạnh để kiểm chứng chính xác mà không cần huấn luyện. Hình 12.1 so sánh cách tiếp cận sinh này với các bộ kiểm chứng phân biệt truyền thống.

Dù các lời phê bình bằng ngôn ngữ tự nhiên hữu ích cho con người, một số thuật toán tìm kiếm (như tìm kiếm cây) có thể cần điểm số dạng số để xếp hạng các đường lập luận khác nhau. Để trích xuất một điểm số liên tục từ bộ kiểm chứng sinh, một phương pháp phổ biến là đo xác suất mô hình đưa ra một token chỉ thị (như "Correct" hay "Incorrect", tức "Đúng" hay "Sai") làm từ đầu tiên của lời phê bình. Do đó, điểm kiểm chứng dạng



(a) Discriminative Verifier (b) Generative Verifier

Hình 12.1: So sánh giữa các cách tiếp cận kiểm chứng phân biệt và kiểm chứng sinh. Hình con (a) cho thấy một bộ kiểm chứng phân biệt, gắn một đầu phân loại vào tầng ẩn cuối cùng của LLM để đưa ra một xác suất đúng dưới dạng số. Hình con (b) cho thấy một bộ kiểm chứng sinh, prompt LLM với truy vấn $\mathbf{z}$, lịch sử lập luận $\bar{\mathbf{y}}_{\leq t}$ và một chỉ dẫn $\mathbf{c}$ để sinh ra một lời phê bình bằng ngôn ngữ tự nhiên thay vì một giá trị vô hướng. (Nhãn trong hình: *Discriminative/Generative Verifier* = bộ kiểm chứng phân biệt/sinh; *last hidden layer* = tầng ẩn cuối; *classification head* = đầu phân loại; *Probability of “Correct”* = xác suất “Đúng”; *Critique in text* = lời phê bằng văn bản.)

số có thể được định nghĩa là:

$$v(\mathbf{z}, \bar{\mathbf{y}}_{\leq t}, \mathbf{c}) \;=\; \Pr(y_{\text{anchor}} = \text{“Correct”} | \mathbf{z}, \bar{\mathbf{y}}_{\leq t}, \mathbf{c}). \tag{12.14}$$

Ở đây y_{anchor} là token chỉ thị, thường được trích xuất từ văn bản đầu ra bằng các heuristic.

Chú giải của người dịch: Biến lời phê thành điểm số

Ở vị trí từ đầu tiên của lời phê, LLM tính một phân phối xác suất trên toàn bộ từ vựng. Giả sử nó gán $\Pr(\text{“Correct”}) = 0.8$ và $\Pr(\text{“Incorrect”}) = 0.15$; khi đó điểm kiểm chứng là 0.8. Một bước khác có $\Pr(\text{“Correct”}) = 0.3$ sẽ bị xếp hạng thấp hơn. Như vậy ta tận dụng “độ tự tin” của mô hình mà không cần huấn luyện thêm đầu phân loại nào.

Bộ kiểm chứng sinh có một số ưu điểm so với các mô hình phần thưởng vô hướng truyền thống. Thứ nhất, chúng mang lại khả năng diễn giải cho việc giải quyết vấn đề. Vì bộ kiểm chứng giải thích lập luận của mình bằng văn bản, các nhà phát triển có thể chẩn đoán vì sao một đường lập luận được chấp nhận hay không. Thứ hai, chúng cho phép tự sửa lỗi (*self-correction*). Vì lời phê bình bằng văn bản thường nói rõ cách sửa một lỗi (như đưa ra khai triển đúng $2y + 4$ trong ví dụ của chúng ta), hệ thống lập luận có thể trực tiếp tích hợp phản hồi này để sửa trạng thái hiện tại và tiếp tục tìm kiếm, thay vì đơn giản loại bỏ toàn bộ đường. Thứ ba, chúng đòi hỏi sửa đổi kiến trúc tối thiểu. Miễn là các LLM hiện tại có năng lực lập luận mạnh, chính cùng một mô hình có thể được dùng vừa làm bộ giải bài toán vừa làm bộ kiểm chứng.

3. Hệ thống ký hiệu hình thức làm bộ kiểm chứng

Dù các bộ kiểm chứng nơ-ron được dùng rộng rãi, chúng có chung một điểm yếu: chúng vẫn là mạng nơ-ron. Điều này nghĩa là chúng cũng có thể bị ảo giác hoặc mắc lỗi logic. Với những nhiệm vụ đòi hỏi độ chính xác nghiêm ngặt, như toán học phức tạp hay kỹ nghệ phần mềm, ta cũng có thể dùng các hệ thống ký hiệu hình thức (*formal symbolic systems*) làm bộ kiểm chứng. Chẳng hạn, trong các nhiệm vụ tối ưu hóa mã, khi mô hình cố gắng tối ưu mã biểu diễn trung gian (*intermediate representation*) (như LLVM-IR), một bộ kiểm tra tương đương hình thức (*formal equivalency checker*) (như Alive2) có thể tự động kiểm chứng xem mã mới sinh có giữ nguyên chính xác ngữ nghĩa như mã gốc hay không.

Các hệ thống hình thức tuân thủ nghiêm ngặt những quy tắc toán học và logic được định nghĩa trước, nên phán quyết của chúng đáng tin cậy 100%. Để dùng một hệ thống hình thức làm bộ kiểm chứng, LLM phải sinh ra các bước lập luận dưới dạng mã thực thi được hoặc các mệnh đề toán học hình thức, thay vì chỉ là ngôn ngữ tự nhiên thông thường. Ví dụ, khi giải một bài toán, LLM có thể viết một đoạn mã Python nhỏ để thực hiện một phép tính trung gian cụ thể. Hệ thống sau đó thực thi đoạn mã này bằng một công cụ bên ngoài. Quá trình kiểm chứng rất đơn giản:

- Nếu mã chạy không lỗi và cho ra một đầu ra hợp lệ, bước lập luận được xác nhận là đúng.

- Nếu mã báo lỗi (chẳng hạn lỗi cú pháp hay vi phạm toán học), bước lập luận được xác nhận là sai.

Một cách hình thức, gọi một bước trung gian $\bar{\mathbf{y}}_t$ là một khối mã hoặc logic hình thức được sinh ra, và gọi E là môi trường thực thi hình thức. Điểm kiểm chứng có thể được định nghĩa là:

$$v(\mathbf{x}, \bar{\mathbf{y}}_{<t}, \bar{\mathbf{y}}_t) \;=\; \begin{cases} 1, & \text{nếu } E(\bar{\mathbf{y}}_t) \text{ trả về kết quả đúng,} \\ 0, & \text{trường hợp khác.} \end{cases} \tag{12.15}$$

Cách tiếp cận này có một ưu điểm rõ ràng: nó loại bỏ các "dương tính giả" gây ra bởi ảo giác của bộ kiểm chứng. Hơn nữa, khi thực thi thất bại, hệ thống hình thức thường sinh ra một thông báo lỗi. Thông báo lỗi này có thể được đưa trở lại LLM để giúp mô hình hiểu điều gì đã sai và sửa logic của nó ở lần thử tiếp theo.

Tuy nhiên, dùng các hệ thống ký hiệu hình thức cũng có những hạn chế. Thứ nhất, nó đòi hỏi LLM có kỹ năng lập trình hoặc hình thức hóa tốt, vì việc chuyển lập luận trừu tượng thành mã nghiêm ngặt là khó. Thứ hai, dù các hệ thống hình thức rất phù hợp cho toán học và lập trình, chúng không phù hợp với các nhiệm vụ mở trong thế giới thực (như viết một bài luận hay lên kế hoạch một chuyến đi), nơi lập luận không thể dễ dàng ánh xạ thành các quy tắc logic cứng nhắc.

Chú giải của người dịch: Một ví dụ về kiểm chứng bằng thực thi mã

Với bài toán của Tom, mô hình có thể viết bước trung gian thành mã: `total = 3*40 + 2*20; print(total*0.9)`. Máy tính chạy đoạn mã và in ra `144.0`. Kết quả do máy tính thực hiện thì không thể "ảo giác", nên nếu mô hình trước đó viết bằng lời rằng đáp số là $146, ta biết ngay đó là sai. Đây cũng là lý do nhiều hệ thống hiện nay cho LLM "dùng máy tính bỏ túi" (công cụ thực thi mã) thay vì tự nhẩm.

12.4 Học cách lập luận

Cải thiện khả năng lập luận (*reasoning*) của các LLM là một việc khó, và vì vậy thường đòi hỏi kết hợp nhiều phương pháp. Như đã bàn ở mục trước, co giãn lúc suy luận (*inference-time scaling*) nhằm kích hoạt quá trình "suy nghĩ" của các LLM có sẵn trong lúc sinh văn bản. Một hướng khác, rất tự nhiên, là nội tại hóa các khả năng lập luận này trực tiếp vào tham số của mô hình trong quá trình huấn luyện. Trong mục này, chúng ta sẽ tập trung vào mô thức "học cách lập luận" đó.

Dữ liệu lập luận có thể được đưa vào bất kỳ giai đoạn huấn luyện nào. Ví dụ, trong giai đoạn tiền huấn luyện (*pre-training*), các đường lập luận có thể đơn giản được thêm vào như văn bản bình thường cho bài toán dự đoán token tiếp theo tiêu chuẩn. Ta cũng có thể xét việc huấn luyện dựa trên lập luận ở giai đoạn hậu huấn luyện (*post-training*). Chẳng hạn, trong tinh chỉnh có giám sát (*supervised fine-tuning*), mô hình có thể được tinh chỉnh trên những mẫu chất lượng cao về cách giải bài toán từng bước. Điều này thường được thực hiện bằng cách huấn luyện mô hình trên các tập dữ liệu chứa các quỹ đạo (*trajectory*) CoT. Hơn nữa, học tăng cường (*reinforcement learning*) có thể được áp dụng để tối ưu thêm các quá trình lập luận này. Ở đây chúng ta tập trung vào việc đưa sự giám sát dựa trên lập luận vào quá trình căn chỉnh (*alignment*) LLM. Trong các tiểu mục sau, chúng ta sẽ bàn về dữ liệu lập luận có giám sát, giám sát quá trình, các kỹ thuật học tăng cường, và các chiến lược để co giãn khả năng lập luận.

12.4.1 Dữ liệu lập luận có giám sát

Một cách hình thức, trong tinh chỉnh có giám sát, một LLM được huấn luyện trên một tập dữ liệu gồm các cặp đầu vào–đầu ra $\mathcal{D} = \{(\mathbf{x}, \mathbf{y})\}$. Hàm mục tiêu huấn luyện tiêu chuẩn là

$$\hat{\theta} \;=\; \arg\max_{\theta} \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}} \log \Pr_\theta(\mathbf{y}|\mathbf{x}), \tag{12.16}$$

trong đó $\Pr_\theta(\cdot)$ ký hiệu mô hình được tham số hóa bởi θ.

Nếu $\mathbf{y}$ chỉ đơn thuần là kết luận cuối cùng, mô hình sẽ học cách dự đoán câu trả lời mà không sinh ra một cách tường minh đường lập luận tương ứng. Với huấn luyện/tinh chỉnh dựa trên lập luận, ta định nghĩa lại $\mathbf{y}$ là một đường lập luận. Như trong Phương trình (12.2), ta có thể biểu diễn đầu ra như một đường lập luận gồm một chuỗi các bước lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_p}\}$. Do đó, ta có thể viết lại hàm mục tiêu trên thành

$$\hat{\theta} \;=\; \arg\max_{\theta} \sum_{(\mathbf{x},\{\bar{\mathbf{y}}_1,...,\bar{\mathbf{y}}_{n_p}\})\in\mathcal{D}} \log \Pr_\theta(\bar{\mathbf{y}}_1, ..., \bar{\mathbf{y}}_{n_p}|\mathbf{x}). \tag{12.17}$$

Bằng cách này, mô hình được cung cấp tường minh các chi tiết lập luận và được huấn luyện để sinh ra một đường lập luận hoàn chỉnh, kể cả kết luận cuối cùng.

Chú giải của người dịch: Đọc Phương trình (12.16)–(12.17)

$\log \Pr_\theta(\mathbf{y}|\mathbf{x})$ đo mức "mô hình tin" rằng đầu ra đúng là $\mathbf{y}$ khi gặp đầu vào $\mathbf{x}$; xác suất càng gần 1 thì logarit càng gần 0 (lớn nhất). Ký hiệu $\arg\max_\theta$ nghĩa là "tìm bộ tham số θ làm cho tổng phía sau lớn nhất". Hai phương trình chỉ khác nhau ở *cái mà mô hình phải tạo ra*: ở (12.16) có thể chỉ là đáp số "12", còn ở (12.17) là toàn bộ lời giải từng bước rồi mới đến đáp số. Giống như khi chấm bài, giáo viên yêu cầu học sinh trình bày cả lời giải chứ không chỉ ghi đáp số.

Trong thực tế, để tách quá trình lập luận cốt lõi khỏi các phần khác của đầu ra, các mô hình lập luận thường dùng những token định dạng đặc biệt. Một cách làm được dùng rộng rãi là bọc các bước lập luận cốt lõi bên trong cặp thẻ `<think>` và `</think>`. Khi một LLM sinh văn bản theo định dạng này, các bước lập luận viết bên trong cặp thẻ được gọi là một **vết CoT** (*CoT trace*) (hay đơn giản là một **vết lập luận** (*reasoning*

trace)). Ta có thể coi vết CoT này như tờ *giấy nháp* của LLM. Giống như một học sinh làm bài thi toán ghi các phép tính trung gian ra giấy nháp trước khi viết đáp số cuối cùng vào tờ bài thi chính thức, LLM dùng khối `<think>` để "trình bày cách làm".

Ví dụ, một mẫu huấn luyện được định dạng bằng các thẻ này có thể trông như sau:

15% của 80 là bao nhiêu?
`<think>`
Để tính cho dễ, tôi có thể tách 15% thành 10% và 5%.
Trước hết, 10% của 80 là 8.
Tiếp theo, 5% bằng một nửa của 10%, tức là 4.
Cộng lại: $8 + 4 = 12$.
`</think>`
Câu trả lời là 12.

Ở đây truy vấn là "15% của 80 là bao nhiêu?". Đầu ra có năm bước, trong đó các bước 1–4 tạo thành vết CoT và bước 5 là kết luận. Cả hai thẻ `<think>` và `</think>` đều là token hợp lệ, nên mô hình sẽ học cách sinh ra chúng để đánh dấu các bước lập luận cốt lõi. Ở phía người dùng, ta có thể xóa các thẻ này để hiển thị một đầu ra gọn gàng. Lưu ý rằng trong các ứng dụng thực tế, đầu ra có thể rất phức tạp và chứa cả nội dung không liên quan đến lập luận. Vì vậy, việc đưa vào cặp thẻ `<think>` và `</think>` có thể giúp mô hình tập trung vào việc giải bài toán lập luận.

Dữ liệu lập luận cũng có thể được biểu diễn bằng nhiều định dạng khác. Một lựa chọn đơn giản là định dạng từng bước bằng ngôn ngữ tự nhiên. Thay vì dùng token đặc biệt, dữ liệu dựa vào các tiền tố văn bản như "Bước 1:", "Bước 2:", hay đơn giản là "Suy nghĩ:" rồi sau đó là "Trả lời:". Ngoài ra, dữ liệu lập luận có thể được cấu trúc bằng cách tách thông điệp theo vai trò. Trong các khuôn mẫu hội thoại (*chat template*) hiện đại, thay vì trộn lập luận và câu trả lời trong một thông điệp duy nhất, vết lập luận được gán cho một vai trò nội bộ (ví dụ vai trò "thought" hay "system_scratchpad"), còn kết luận cuối cùng được gán cho vai trò "assistant" tiêu chuẩn. Cách này tách phần suy nghĩ nội bộ của LLM khỏi phản hồi hướng tới người dùng ngay ở cấp hệ thống.

12.4.2 Giám sát quá trình

Như đã bàn ở Mục 12.3.4, các mô hình phần thưởng quá trình (*process reward model, PRM*) (PRM) là công cụ mạnh để kiểm chứng các bước lập luận trong lúc suy luận. Một mở rộng tự nhiên là dùng chúng để định hướng việc huấn luyện LLM thông qua giám sát quá trình (*process supervision*).

Trong giám sát kết quả (*outcome supervision*) tiêu chuẩn (chẳng hạn tinh chỉnh có giám sát), mô hình chỉ được đánh giá dựa trên kết quả cuối cùng của toàn bộ đường lập luận $\mathbf{y}$. Nếu mô hình sinh ra một chuỗi năm bước và mắc lỗi logic ở bước thứ ba, một cách huấn luyện dựa trên kết quả có thể chỉ đơn giản phạt toàn bộ chuỗi. Mô hình phải tự đoán xem chính xác bước nào gây ra thất bại. Đây được gọi là **bài toán phân bổ công trạng** (*credit assignment problem*) [Minsky, 1961; Sutton, 1984].

Giám sát quá trình giải quyết vấn đề này bằng cách áp dụng tín hiệu huấn luyện ở cấp từng bước. Thay vì một phần thưởng duy nhất cho cả đường lập luận, mô hình nhận phản hồi cho mỗi bước riêng lẻ $\bar{\mathbf{y}}_t$. Ví dụ, hãy tưởng tượng bạn dạy một học sinh giải một phương trình đại số phức tạp. Nếu bạn chỉ nhìn con số cuối cùng em viết ra và nói "Sai", học sinh sẽ không biết mình hiểu sai công thức, cộng nhầm, hay quên một dấu trừ. Giám sát quá trình giống như việc giáo viên chỉ đích danh dòng thứ 3 và nói "Lập luận của em hỏng ở chỗ này".

Giám sát quá trình thường được đặt dưới dạng một bài toán học tăng cường. Để áp dụng học tăng cường cho giám sát quá trình, ta thường theo một vòng lặp huấn luyện gồm ba bước:

- **Triển khai (rollout)**: LLM nhận một đầu vào $\mathbf{x}$ và sinh ra một đường lập luận đầy đủ $\mathbf{y} = \{\bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_{n_p}\}$. Quá trình triển khai có thể được lặp lại nhiều lần để sinh ra một tập các đường lập luận $\mathcal{Y}$.
- **Kiểm chứng (verification)**: Một mô hình phần thưởng (tức là một PRM kiểm tra các bước trung gian) đánh giá từng đường lập luận $\mathbf{y}$ và trả về một điểm phần thưởng $R(\mathbf{x}, \mathbf{y})$.
- **Cập nhật (update)**: Một thuật toán học tăng cường dùng các phần thưởng này để cập nhật tham số θ của mô hình.

Một hàm mục tiêu huấn luyện đơn giản có thể được phát biểu là cực đại

hóa phần thưởng kỳ vọng

$$J(\theta) \;=\; \mathbb{E}_{\mathbf{y}\sim \mathrm{Pr}_\theta(\cdot|\mathbf{x})}\big[R(\mathbf{x},\mathbf{y})\big]. \tag{12.18}$$

Ta có thể sửa đổi hàm mục tiêu này để huấn luyện ổn định hơn, chẳng hạn đưa hàm ưu thế (*advantage function*) vào hàm mục tiêu như trong PPO.

Chú giải của người dịch: Phần thưởng kỳ vọng

Ký hiệu $\mathbb{E}_{\mathbf{y}\sim \mathrm{Pr}_\theta(\cdot|\mathbf{x})}[\,\cdot\,]$ là "giá trị trung bình khi $\mathbf{y}$ được lấy ngẫu nhiên theo cách mô hình sinh ra". Ví dụ, nếu với một bài toán mô hình sinh lời giải đúng (phần thưởng 1) với xác suất 0.7 và lời giải sai (phần thưởng 0) với xác suất 0.3, thì phần thưởng kỳ vọng là $0.7 \cdot 1 + 0.3 \cdot 0 = 0.7$. Huấn luyện nhằm điều chỉnh θ để con số này tăng lên, tức là mô hình ngày càng hay sinh ra lời giải được thưởng cao.

Dù điều này trông giống quy trình học tăng cường tiêu chuẩn dùng trong căn chỉnh LLM nói chung (như mô tả ở Chương 10), có một khác biệt căn bản về bản chất của phần thưởng. Trong các tác vụ căn chỉnh LLM thông thường, mô hình phần thưởng thường đánh giá toàn bộ văn bản được sinh ra và trả về một phần thưởng thưa (*sparse reward*) ở ngay cuối cùng. Khi đó thuật toán học tăng cường gặp khó khăn với bài toán phân bổ công trạng, vì nó phải đoán xem những token hay đoạn nào thực sự đóng góp vào điểm số cuối cùng đó.

Ngược lại, lập luận diễn ra từng bước một. Do đó, điểm kiểm chứng, mà để đơn giản ta ký hiệu là $v_t = v(\mathbf{x}, \bar{\mathbf{y}}_{\le t})$, vốn dĩ là một tín hiệu ở cấp bước. Đặc điểm này khiến các tác vụ lập luận rất phù hợp với học tăng cường hiện đại. Vì v_t được tạo ra theo từng bước, nó tự nhiên biến một tác vụ sinh ngôn ngữ có phần thưởng thưa thành một môi trường có phần thưởng dày (*dense reward*).

Câu hỏi kỹ thuật chính khi đó trở thành làm sao biến đổi đúng cách các điểm cấp bước tự nhiên v_t này thành những tín hiệu huấn luyện hình thức (như phần thưởng từng bước có cấu trúc r_t hay ưu thế A_t) mà các thuật toán đòi hỏi. Tùy theo định dạng đầu ra của PRM và khung học tăng cường, ta xét ba phương pháp sau.

1. Huấn luyện với điểm kiểm chứng nhị phân

Trong một số trường hợp, PRM đóng vai trò một bộ kiểm tra quy tắc nghiêm ngặt (như một máy tính toán học hay một trình biên dịch mã). Nó chỉ trả về một điểm nhị phân: $v_t \in \{0, 1\}$. Điểm 1 nghĩa là logic ở bước t là đúng khi biết các bước trước đó, còn 0 nghĩa là có lỗi logic xảy ra ở bước này.

Khi làm việc với điểm nhị phân, ý tưởng rất đơn giản: chỉ một bước sai cũng làm hỏng toàn bộ đường lập luận. Nếu muốn gán một phần thưởng cấp chuỗi $R(\mathbf{x}, \mathbf{y})$ cho cả đường lập luận, ta chỉ cần nhân tất cả điểm của các bước với nhau

$$R(\mathbf{x}, \mathbf{y}) \;=\; \prod_{t=1}^{n_p} v_t. \tag{12.19}$$

Như vậy mô hình chỉ nhận phần thưởng dương nếu điểm kiểm chứng của mọi bước đều bằng 1.

Một cách khác, nếu thuật toán học tăng cường hỗ trợ phần thưởng cấp bước (ký hiệu là r_t), ta có thể phạt mô hình đúng tại chỗ xảy ra thất bại. Ví dụ, ta có thể cho phần thưởng 1 chừng nào các bước vẫn còn đúng ($v_t = 1$). Ngay khi mô hình mắc lỗi ($v_t = 0$), ta gán một phần thưởng âm và dừng việc sinh.

2. Huấn luyện với điểm kiểm chứng giá trị thực

Trong hầu hết các PRM nơ-ron, bộ kiểm chứng xuất ra một số thực liên tục $v_t \in \mathbb{R}$. Đó có thể là một xác suất từ 0 đến 1 (thể hiện độ tin cậy rằng đường lập luận là đúng) hoặc một giá trị vô hướng chưa chuẩn hóa.

Để áp dụng điểm kiểm chứng liên tục, kỹ thuật phổ biến nhất là **định hình phần thưởng** (*reward shaping*). Thay vì dùng điểm tuyệt đối, ta định nghĩa phần thưởng tức thời của bước r_t là hiệu số biên giữa bước hiện tại và bước trước đó:

$$r_t \;=\; v_t - v_{t-1}. \tag{12.20}$$

Phương pháp này rất trực quan. Hãy tưởng tượng một PRM ước lượng xác suất đi đến đáp án cuối cùng đúng. Nếu Bước 1 có độ tin cậy 0.4, và Bước 2 chứa một suy diễn xuất sắc nâng độ tin cậy lên 0.9, thì phần thưởng cho Bước 2 là $r_2 = 0.9 - 0.4 = +0.5$. Mô hình được thưởng cho chính bước nhảy logic đó. Nếu Bước 3 đưa vào một lỗi tính toán và độ tin cậy tụt xuống 0.1, phần thưởng là $r_3 = 0.1 - 0.9 = -0.8$.

Khi cộng các phần thưởng đã định hình này trên toàn bộ chuỗi, ta được một "tổng rút gọn kiểu ống lồng (*telescoping sum*)" tạo thành phần thưởng cấp chuỗi $R(\mathbf{x}, \mathbf{y})$:

$$\begin{aligned} R(\mathbf{x}, \mathbf{y}) &= \sum_{t=1}^{n_p} r_t \\ &= (v_1 - v_0) + (v_2 - v_1) + \cdots + (v_{n_p} - v_{n_p-1}) \\ &= v_{n_p} - v_0. \end{aligned} \tag{12.21}$$

Ở đây, v_0 thường được giả định bằng 0. Trong phép biến đổi này, các điểm trung gian triệt tiêu lẫn nhau. Điều đó có nghĩa là phần thưởng tổng thể của chuỗi vẫn nhất quán với đánh giá của bước cuối cùng.

Chú giải của người dịch: Tổng "ống lồng"

Đây chính là kiểu tổng bạn có thể đã gặp khi tính $\frac{1}{1\cdot 2} + \frac{1}{2\cdot 3} + \cdots$ bằng cách tách $\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1}$: các số hạng ở giữa khử nhau, chỉ còn đầu và cuối. Với ví dụ trên ($v_0 = 0$, $v_1 = 0.4$, $v_2 = 0.9$, $v_3 = 0.1$): $r_1 = 0.4$, $r_2 = +0.5$, $r_3 = -0.8$, tổng là $0.1 = v_3 - v_0$. Như vậy định hình phần thưởng chỉ "chia nhỏ" điểm cuối cùng ra cho từng bước, chứ không làm thay đổi tổng.

Để hiểu phần thưởng dày từng bước này thực sự huấn luyện mô hình như thế nào, ta cần nối nó với hàm mục tiêu học tăng cường. Như trong Phương trình (12.18), mục tiêu của ta là cực đại hóa tổng phần thưởng kỳ vọng trên mọi đường lập luận có thể được sinh ra. Để huấn luyện hiệu quả và phân bổ công trạng hợp lý, ta thường dùng *nguyên lý nhân quả*: một hành động thực hiện ở bước t chỉ có thể ảnh hưởng tới các phần thưởng nhận được sau bước này. Vì vậy, thay vì đánh giá một bước bằng phần thưởng của cả chuỗi, ta đánh giá nó bằng lợi tức tương lai (*future return*) R_t, được định nghĩa là tổng các phần thưởng từ bước t tới cuối chuỗi:

$$\begin{aligned} R_t &= \sum_{k=t}^{n_p} r_k \\ &= v_{n_p} - v_{t-1}. \end{aligned} \tag{12.22}$$

Cuối cùng, ta có thể huấn luyện mô hình bằng hàm mất mát thay thế

(*surrogate loss function*)

$$\mathcal{L}(\theta) \;=\; -\sum_{t=1}^{n_p} \log \Pr_\theta(\bar{\mathbf{y}}_t | \mathbf{x}, \bar{\mathbf{y}}_{<t}) \cdot R_t. \tag{12.23}$$

Hàm mất mát này cung cấp một cơ chế phản hồi ở cấp bước. Với bất kỳ đường lập luận nào được sinh ra, mô hình nhìn vào lợi tức tương lai R_t tại mỗi bước t. Nếu R_t dương, hàm mất mát đẩy mô hình tăng xác suất $\log \Pr_\theta(\cdot)$ sinh ra bước này. Nếu R_t âm, nó đóng vai trò một hình phạt, đẩy mô hình tránh sinh ra bước này trong tương lai.

Chú giải của người dịch: Vì sao có dấu trừ trong (12.23)?

Khi huấn luyện, ta luôn *cực tiểu hóa* hàm mất mát. Cực tiểu $-\sum \log \Pr \cdot R_t$ cũng chính là cực đại $\sum \log \Pr \cdot R_t$. Mỗi số hạng giống một "trọng số khuyến khích": bước nào có $R_t = +0.5$ thì mô hình được đẩy tăng xác suất sinh ra bước đó; bước nào có $R_t = -0.8$ thì ngược lại, xác suất bị đẩy giảm, và đẩy mạnh hơn vì $|R_t|$ lớn hơn.

3. Đưa kết quả kiểm chứng vào hàm ưu thế

Nhiều thuật toán học tăng cường (như PPO) tối ưu mô hình bằng hàm ưu thế A_t, hàm này đo một hành động tốt hơn bao nhiêu so với mức được kỳ vọng. Thay vì xem điểm kiểm chứng như phần thưởng thô, ta có thể đưa chúng vào phép tính ưu thế.

Một phương pháp là đưa kiểm chứng cấp bước vào sai số chênh lệch thời gian (*temporal difference error*). Nếu PRM được huấn luyện để ước lượng phần thưởng tương lai kỳ vọng, ta có thể xem nó như giá trị cơ sở (*value baseline*) trong học tăng cường tác nhân–phê bình (*actor-critic*) tiêu chuẩn. Ta tính ưu thế A_t bằng sai số chênh lệch thời gian

$$A_t \;=\; \tilde{r}_t + \gamma v_t - v_{t-1}, \tag{12.24}$$

trong đó $\tilde{r}_t$ là phần thưởng bên ngoài bất kỳ đến từ môi trường (thường bằng 0 ở các bước suy nghĩ trung gian) và γ là hệ số chiết khấu (*discount factor*).

Một phương pháp khác là cải tiến cách thiết kế ưu thế tương đối theo nhóm. Trong **tối ưu chính sách tương đối theo nhóm** (*group relative policy optimization*) (**GRPO**), ta bỏ hẳn các điểm kiểm chứng tuyệt đối

[Shao và cộng sự, 2024]. Thay vào đó, ta tính ưu thế bằng cách so sánh một nhóm các đường lập luận. Với một đầu vào $\mathbf{x}$, LLM sinh ra một nhóm các đường lập luận khác nhau. Một bộ kiểm chứng gán điểm $v_t^{(i)}$ cho bước thứ t của đường thứ i. Ưu thế $A_t^{(i)}$ được tính bằng cách cộng dồn các điểm đã chuẩn hóa từ bước t tới cuối đường lập luận như sau

$$A_t^{(i)} = \sum_{k=t}^{n_p^{(i)}} \tilde{v}_k^{(i)}, \tag{12.25}$$

trong đó $n_p^{(i)}$ là tổng số bước của đường thứ i, và $\tilde{v}_k^{(i)}$ là điểm đã chuẩn hóa, được định nghĩa là:

$$\tilde{v}_k^{(i)} = \frac{v_k^{(i)} - \mu}{\sigma}, \tag{12.26}$$

với μ và σ là trung bình và độ lệch chuẩn của các điểm trên tất cả các bước của các đường lập luận trong nhóm. Ví dụ, nếu một mô hình thử 5 cách khác nhau để giải một phương trình, và đường số 3 thực hiện một bước có điểm cao hơn đáng kể so với trung bình của mọi bước trong các đường này, thì đường số 3 nhận ưu thế dương. Ở đây, trung bình nhóm μ đóng vai trò giá trị cơ sở, nhờ đó không cần huấn luyện và chạy một mạng giá trị (*value network*) riêng.

Chú giải của người dịch: Chuẩn hóa điểm trong GRPO

Phép tính (12.26) chính là "điểm chuẩn" (z-score) trong thống kê mô tả: lấy điểm trừ trung bình rồi chia cho độ lệch chuẩn. Ví dụ nhóm có các điểm bước 0.2, 0.4, 0.6, 0.8: trung bình $\mu = 0.5$, độ lệch chuẩn $\sigma \approx 0.224$. Bước có điểm 0.8 được $\tilde{v} = (0.8 - 0.5)/0.224 \approx +1.34$ (tốt hơn mặt bằng chung), bước có điểm 0.2 được ≈ -1.34. Như vậy điều quan trọng không phải điểm tuyệt đối, mà là bước đó tốt hơn hay kém hơn *so với các lời giải khác trong cùng nhóm*.

Khi ưu thế A_t (hoặc $A_t^{(i)}$) đã được tính bằng một trong hai phương pháp trên, nó có thể được dùng làm trọng số co giãn trong hàm mất mát. Một phiên bản đơn giản hóa của hàm mất mát dựa trên ưu thế có thể viết

là[2]:

$$\mathcal{L}(\theta) \;=\; -\sum_{t=1}^{n_p} \log \Pr_\theta(\bar{\mathbf{y}}_t | \mathbf{x}, \bar{\mathbf{y}}_{<t}) \cdot A_t. \tag{12.27}$$

Lưu ý rằng lợi ích của việc dùng ưu thế A_t thay cho lợi tức thô R_t là giảm phương sai. Bằng cách trừ đi một giá trị cơ sở, dù đó là giá trị ước lượng v_{t-1} hay trung bình nhóm μ, hàm ưu thế cung cấp một thước đo hiệu năng *tương đối*. Nó cho mô hình biết không chỉ một bước lập luận có hợp logic hay không, mà còn liệu bước đó có tốt hơn mức kỳ vọng hay không. Phản hồi tương đối này ngăn mô hình bị "choáng ngợp" bởi các truy vấn quá dễ hoặc quá khó.

12.4.3 Chưng cất on-policy cho lập luận

Dù giám sát quá trình cung cấp tín hiệu phần thưởng từng bước để định hướng lập luận, nó vẫn dựa vào một giá trị vô hướng để cho mô hình biết nó đang làm tốt đến mức nào. Để cung cấp một tín hiệu huấn luyện còn phong phú hơn, ta có thể dùng **chưng cất on-policy** (*on-policy distillation*) (**OPD**). Phương pháp này thường được dùng trong hậu huấn luyện LLM, và rất phù hợp với mục tiêu của giám sát theo bước: làm cho tín hiệu huấn luyện dày đặc nhất có thể.

Chưng cất tri thức (*knowledge distillation*) tiêu chuẩn có thể được xem là một dạng **chưng cất off-policy** (*off-policy distillation*). Một mô hình "học sinh" nhỏ hơn học bằng cách bắt chước các đường lập luận do một mô hình "giáo viên" mạnh hơn sinh ra. Học sinh được huấn luyện trên một tập dữ liệu cố định gồm các đường lập luận ghi sẵn này. Tuy nhiên, lập luận là một quá trình dài và dễ đổ vỡ. Nếu học sinh mắc một lỗi nhỏ hoặc chọn một hướng hợp lệ hơi khác trong lúc sinh thực tế, nó sẽ rơi vào một trạng thái chưa từng gặp trong dữ liệu huấn luyện. Vì học sinh không biết cách gượng dậy sau lỗi của mình, quá trình lập luận có thể sụp đổ. Hiện tượng này được gọi là thiên lệch phơi nhiễm (*exposure bias*) hay dịch chuyển phân phối (*distribution shift*).

[2]Lưu ý rằng các thuật toán nâng cao như PPO và GRPO thường tăng cường hàm mục tiêu này bằng cách thêm một cơ chế *cắt* (*clipping*) (giới hạn tỉ số xác suất giữa chính sách mới và chính sách cũ) để ngăn tham số mô hình thay đổi quá mạnh trong một lần cập nhật. Tuy nhiên, nguyên lý toán học cốt lõi vẫn như vậy.

Chú giải của người dịch: On-policy và off-policy

Trong học tăng cường, "chính sách" (*policy*) là cách mô hình hiện tại ra quyết định, ở đây là cách nó sinh từng bước. *On-policy* nghĩa là học từ dữ liệu do chính mô hình đang được huấn luyện sinh ra; *off-policy* là học từ dữ liệu do một "người khác" (giáo viên, con người, hay phiên bản cũ) sinh ra. Giống như học lái xe: xem video người khác lái (off-policy) khác với tự cầm lái và được thầy chỉnh ngay khi đi lệch (on-policy).

Ý tưởng cốt lõi của OPD là mô hình học sinh học từ chính các quỹ đạo nó sinh ra, thay vì từ các quỹ đạo được lấy mẫu từ mô hình giáo viên. Một cách hình thức, gọi mô hình giáo viên là $\Pr^t(\cdot)$ và mô hình học sinh (được tham số hóa bởi θ) là $\Pr^s_\theta(\cdot)$. Quy trình OPD điển hình gồm hai bước:

- **Học sinh triển khai.** Cho một đầu vào $\mathbf{x}$, mô hình học sinh dùng tham số hiện tại θ để sinh một đường lập luận $\mathbf{y} = \{\bar{\mathbf{y}}_1, \ldots, \bar{\mathbf{y}}_{n_p}\}$. Vì học sinh tự sinh ra đường này, nó tự nhiên khám phá các hành vi lập luận của chính mình. Điều này bao gồm việc đi chệch một chút, chọn hướng tiếp cận khác, hay thậm chí mắc những lỗi mà giáo viên không bao giờ mắc.

- **Giáo viên hướng dẫn.** Sau đó ta đưa đường lập luận do học sinh sinh ra này vào mô hình giáo viên. Tại mỗi bước t, thay vì đưa ra một phần thưởng vô hướng, mô hình giáo viên tính phân phối xác suất cho bước tiếp theo $\Pr^t(\cdot|\mathbf{x}, \bar{\mathbf{y}}_{<t})$.

Lý tưởng nhất, mô hình học sinh được cập nhật để phân phối xác suất cấp bước của nó khớp với phân phối do mô hình giáo viên sinh ra. Mục tiêu là cực tiểu hóa độ phân kỳ KL (*KL divergence*) kỳ vọng giữa hai phân phối trên các đường lập luận do mô hình học sinh sinh ra:

$$\mathcal{L}_{\text{step}}(\theta) \;=\; \mathbb{E}_{\mathbf{y}\sim \Pr^s_\theta(\cdot|\mathbf{x})}\left[\sum_{t=1}^{n_p} D_{\text{KL}}\big(\Pr^s_\theta(\cdot|\mathbf{x}, \bar{\mathbf{y}}_{<t}) \,\|\, \Pr^t(\cdot|\mathbf{x}, \bar{\mathbf{y}}_{<t})\big)\right]. \tag{12.28}$$

Tuy nhiên, trong thực tế việc tính phân phối cấp bước này là bất khả thi về mặt tính toán. Vì một bước lập luận $\bar{\mathbf{y}}_t$ thường gồm nhiều token, phân phối xác suất trên mọi bước nhiều-token có thể có nằm trong một không gian lớn theo hàm mũ.

Để giải quyết điều này, ta quay về cách tiếp cận OPD tiêu chuẩn, hoạt

động ở cấp token. Bằng cách xem đường lập luận được sinh ra như một chuỗi token $\mathbf{y} = \{y_1, \ldots, y_n\}$, hàm mục tiêu được phát biểu lại để tính độ phân kỳ KL theo từng token trên bộ từ vựng:

$$\mathcal{L}_{\text{token}}(\theta) \;=\; \mathbb{E}_{\mathbf{y}\sim \Pr^{s}_{\theta}(\cdot|\mathbf{x})}\left[\sum_{i=1}^{n} D_{\text{KL}}\big(\Pr^{s}_{\theta}(\cdot|\mathbf{x}, \mathbf{y}_{<i}) \,\|\, \Pr^{t}(\cdot|\mathbf{x}, \mathbf{y}_{<i})\big)\right]. \tag{12.29}$$

Hàm mục tiêu này sau đó được xấp xỉ bằng lấy mẫu. Với một đầu vào $\mathbf{x}$ cho trước, ta chỉ cần thực hiện một lượt triển khai để sinh một (hoặc một lô nhỏ) đường lập luận từ mô hình học sinh, rồi tính độ phân kỳ KL cấp token trên các lượt triển khai được lấy mẫu này.

Chú giải của người dịch: Độ phân kỳ KL và vì sao cấp bước là bất khả thi

$D_{\text{KL}}(P\|Q) = \sum_w P(w) \log \frac{P(w)}{Q(w)}$ đo hai phân phối P, Q "khác nhau bao xa"; nó bằng 0 khi hai phân phối trùng nhau và dương khi khác nhau. Ở cấp token, tổng chạy qua bộ từ vựng (chẳng hạn 100 000 token) — máy tính làm được. Nhưng nếu một bước gồm 20 token thì số bước có thể có là $100\,000^{20} = 10^{100}$, nhiều hơn số nguyên tử trong vũ trụ quan sát được; vì thế không thể liệt kê để tính tổng ở cấp bước.

Hình 12.2 so sánh OPD với tinh chỉnh có giám sát và giám sát quá trình. Để hiểu rõ hơn sự khác biệt giữa các phương pháp này, hãy tưởng tượng một học sinh đang cố giải một bài hình học phức tạp:

- **Tinh chỉnh có giám sát.** Giống như đưa cho học sinh một cuốn sách đầy những lời giải đúng do giáo viên viết để học thuộc. Nếu học sinh quyết định kẻ một đường phụ khác với trong sách, cuốn sách không giúp được gì về cách tiếp tục từ điểm xuất phát mới đó.
- **Giám sát quá trình** (học tăng cường với PRM). Giống như có một người chấm bài nghiêm khắc kiểm tra bài làm của học sinh từng bước và nói "+1 điểm cho bước 1", "-1 điểm cho bước 2". Nó cho học sinh biết mình sai ở đâu, nhưng không cho biết cách sửa hay bước tiếp theo đúng phải là gì.
- **Chưng cất on-policy.** Giống như có một gia sư giỏi ngồi ngay cạnh học sinh. Khi học sinh kẻ đường phụ khác thường đó (một trạng thái không có trong sách chuẩn), gia sư nhìn vào trạng thái hiện tại của

học sinh và nói: "Được rồi, vì em đã kẻ đường ở đây, bước tốt nhất tiếp theo là tính góc này."

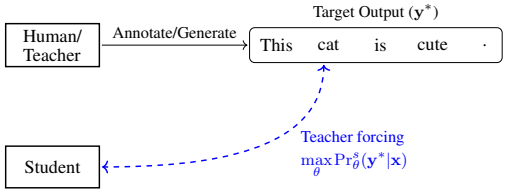
Vì sự giám sát của OPD có dạng một phân phối xác suất đầy đủ trên bộ từ vựng, học sinh nhận được một tín hiệu học rất dày và phong phú. Nó không chỉ học bước tiếp theo tốt nhất là gì, mà còn học cả xác suất của các lựa chọn hợp lệ khác. Phản hồi phong phú này rất thích hợp cho lập luận vì nó dạy mô hình học sinh cách gượng dậy sau khi đi chệch và duy trì tính nhất quán logic trên toàn bộ chuỗi đầu ra, thay vì chỉ xem một lời giải thích như một thành phần tách biệt. Tuy nhiên, áp dụng chưng cất on-policy đi kèm chi phí tính toán cao. Quá trình này đòi hỏi chạy một mô hình giáo viên lớn song song với học sinh trong lúc huấn luyện để tính các phân phối xác suất theo thời gian thực. Điều này tốn kém hơn chưng cất off-policy tiêu chuẩn, nơi đầu ra của mô hình giáo viên chỉ được sinh một lần và lưu trong một tập dữ liệu tĩnh.

Chú giải của người dịch: Đọc nhãn trong Hình 12.2

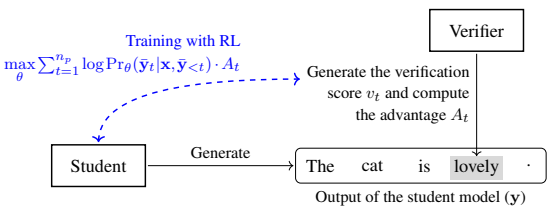
Human/Teacher: người/giáo viên; Annotate/Generate: gán nhãn/sinh ra; Target Output: đầu ra mục tiêu; Student: học sinh; Teacher forcing: ép buộc theo giáo viên (luôn đưa đáp án chuẩn làm ngữ cảnh); Training with RL: huấn luyện bằng học tăng cường; Verifier: bộ kiểm chứng; "Generate the verification score v_t and compute the advantage A_t": sinh điểm kiểm chứng v_t và tính ưu thế A_t; Output of the student model: đầu ra của mô hình học sinh; Minimizing the KL divergence: cực tiểu hóa độ phân kỳ KL; Teacher's/Student's predicted distribution: phân phối dự đoán của giáo viên/học sinh. Câu ví dụ "This cat is cute" / "The cat is lovely" nghĩa là "Con mèo này dễ thương".

12.4.4 Co giãn huấn luyện cho lập luận

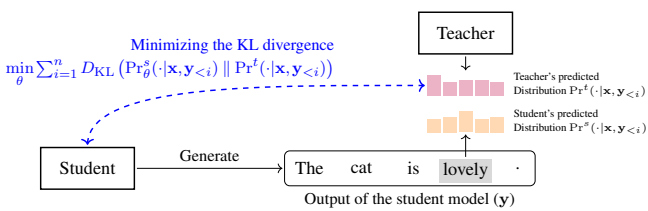
Co giãn là một chiến lược cốt lõi trong kỷ nguyên LLM. Khi xử lý các tác vụ lập luận phức tạp, co giãn quá trình huấn luyện là một ý tưởng tự nhiên. Ở đây, chúng ta bàn ngắn gọn về một số khía cạnh then chốt của việc co giãn huấn luyện cho các LLM lập luận: học tăng cường quy mô lớn, sinh dữ liệu tổng hợp, và co giãn độ dài ngữ cảnh.



(a) Supervised Fine-tuning



(b) Process Supervision



(c) On-policy Distillation

Hình 12.2: So sánh các phương pháp tinh chỉnh có giám sát, giám sát quá trình và chưng cất on-policy. Trong tinh chỉnh có giám sát (a), mô hình học sinh được huấn luyện off-policy để cực đại hóa likelihood của các đầu ra mục tiêu chuẩn do con người hoặc một mô hình giáo viên cung cấp. Trong giám sát quá trình (b), mô hình học sinh tự sinh ra đường lập luận của mình, và một bộ kiểm chứng đánh giá các bước tự sinh này để tính ưu thế. Trong chưng cất on-policy (c), học sinh cũng tự sinh ra đường lập luận của mình, nhưng thay vì phần thưởng vô hướng, một mô hình giáo viên cung cấp một phân phối xác suất dày, cấp token tại mỗi bước. Học sinh được cập nhật bằng cách cực tiểu hóa độ phân kỳ KL giữa phân phối dự đoán của chính nó và phân phối của giáo viên.

1. Học tăng cường quy mô lớn

Học tăng cường đã trở thành một cách tiếp cận chủ đạo để cải thiện khả năng lập luận "Hệ thống 2" của các LLM. Ví dụ, thành công của nhiều mô hình lập luận mạnh đến từ việc mở rộng quy mô học tăng cường chứ không chỉ dùng tinh chỉnh có giám sát. Một hiện tượng thú vị được quan sát trong quá trình huấn luyện dựa trên học tăng cường quy mô lớn là sự xuất hiện của "khoảnh khắc "à há" (*aha moment*)" [DeepSeek, 2025]. Không cần con người gán nhãn tường minh, các LLM học được cách tạm dừng, nhận ra khi mình đã mắc lỗi, hay thậm chí quay lui để thử một hướng khác. Những khả năng này khó khơi gợi được nếu chỉ dựa vào lượng dữ liệu gán nhãn hạn chế và huấn luyện có giám sát tiêu chuẩn. Khi khối lượng tính toán cho huấn luyện tăng lên, mô hình có thể dành những vết CoT dài hơn cho các bài toán khó hơn, và nhờ đó tự khám phá cách suy nghĩ sâu [Snell và cộng sự, 2024].

Chú giải của người dịch: Hệ thống 1 và Hệ thống 2

Đây là cách phân chia của nhà tâm lý học Daniel Kahneman: "Hệ thống 1" là lối nghĩ nhanh, theo trực giác (như trả lời ngay $2 + 2 = 4$); "Hệ thống 2" là lối nghĩ chậm, có chủ đích, từng bước (như giải một bài toán hình học). Các mô hình lập luận cố gắng mô phỏng Hệ thống 2 bằng cách "nghĩ ra giấy nháp" trước khi trả lời.

Các nghiên cứu thực nghiệm quy mô lớn gần đây đã cho thấy mối quan hệ giữa lượng tính toán dành cho học tăng cường và hiệu năng lập luận có thể được mô hình hóa bằng một đường cong hình chữ S [Khatri và cộng sự, 2025; Tan và cộng sự, 2026]. Một phát hiện then chốt từ định luật co giãn (*scaling law*) này là các lựa chọn huấn luyện khác nhau tác động lên đường cong theo những cách khác nhau. Chẳng hạn, tăng năng lực nền của LLM (ví dụ dùng kiến trúc lớn hơn hoặc MoE) và mở rộng độ dài cửa sổ ngữ cảnh có thể nâng giới hạn hiệu năng. Ngược lại, các cải tiến thuật toán, như thay đổi hàm mất mát hay điều chỉnh cách chuẩn hóa ưu thế, chủ yếu giúp tận dụng tốt hơn ngân sách tính toán sẵn có nhưng không cải thiện đáng kể hiệu năng cuối cùng.

Chú giải của người dịch: Đường cong chữ S

Đường cong chữ S (giống đồ thị hàm sigmoid $f(x) = \frac{1}{1+e^{-x}}$) tăng chậm lúc đầu, tăng nhanh ở giữa, rồi chững lại gần một "trần" (đường tiệm cận ngang). Câu trong sách có thể hiểu là: tăng kích thước mô hình hay độ dài ngữ cảnh làm *nâng cái trần* lên; còn các mẹo thuật toán chủ yếu làm đường cong *đi lên nhanh hơn* (đạt gần trần với ít tính toán hơn) mà không đổi trần.

Co giãn học tăng cường là việc khó. Thứ nhất, chi phí tính toán rất cao. Với mỗi mẫu huấn luyện, hệ thống thường thực hiện một lượt triển khai (chạy mô hình để suy luận) rồi gọi thêm một mô hình phần thưởng hoặc một LLM khác để đánh giá đầu ra được sinh ra. Khi số mẫu huấn luyện tăng lên, chi phí này trở nên cao hơn đáng kể so với tinh chỉnh có giám sát tiêu chuẩn. Gánh nặng tính toán này cũng khiến việc chạy thí nghiệm và thử–sai khó khăn hơn nhiều. Hơn nữa, các phương pháp học tăng cường nhìn chung phức tạp và đòi hỏi tinh chỉnh rất cẩn thận để đạt kết quả tốt. Ví dụ, chúng nhạy cảm với thiết lập siêu tham số và dữ liệu huấn luyện. Do đó, các phương pháp học tăng cường khá "mong manh". Khi huấn luyện ở quy mô khổng lồ, việc bảo đảm hệ thống chạy ổn định (chẳng hạn, bảo đảm hàm mất mát huấn luyện giảm đều đặn) không chỉ là một bài toán học máy, mà còn là một thách thức kỹ thuật.

2. Thêm dữ liệu tổng hợp cho lập luận

Vì các chú thích chất lượng cao của con người cho lập luận nhiều bước phức tạp rất khan hiếm, việc co giãn dữ liệu huấn luyện thường dựa vào sinh dữ liệu tổng hợp (*synthetic data generation*). Thay vì viết tay các chứng minh từng bước, các nhà nghiên cứu dùng những LLM mạnh để sinh ra các quỹ đạo lập luận đa dạng, sau đó lọc và chọn lọc để huấn luyện mô hình mới [Zelikman và cộng sự, 2022; Mukherjee và cộng sự, 2023].

Để sinh dữ liệu tổng hợp cho lập luận, một cách làm phổ biến là lấy mẫu nhiều đường lập luận từ các mô hình giáo viên cho một câu hỏi hạt giống cho trước [Wang và cộng sự, 2023c]. Sau đó ta có thể áp dụng các bộ kiểm chứng hoặc môi trường thực thi để đánh giá các đường này, và chỉ giữ lại những đường đi tới đáp án cuối cùng đúng qua các bước logic chặt chẽ. Ngoài ra, có thể dùng mô hình để sinh các biến thể của các bài toán có sẵn (ví dụ thay đổi giá trị số hay ràng buộc cấu trúc trong các tác vụ toán và lập trình) nhằm mở rộng phân phối của dữ liệu huấn luyện.

3. Co giãn độ dài ngữ cảnh

Khi các mô hình lập luận giải quyết những bài toán ngày càng phức tạp, độ dài của các đường lập luận đã trở thành một nút thắt cổ chai chính. Nhu cầu về cửa sổ ngữ cảnh lớn trong lập luận đến từ nhiều nguồn. Thứ nhất, để khơi gợi lối nghĩ sâu kiểu Hệ thống 2, bản thân prompt đầu vào thường cần ngữ cảnh rất dài. Điều này gồm các ràng buộc logic chi tiết, các quy tắc môi trường phức tạp, hay nhiều ví dụ CoT few-shot dài. Thứ hai, quá trình lập luận được sinh ra cho một tác vụ phức tạp có thể dễ dàng kéo dài hàng nghìn, thậm chí hàng chục nghìn token trước khi đi tới kết luận cuối cùng. Thứ ba, những tiến bộ gần đây về tác tử (*agent*) LLM cũng làm tăng nhu cầu lập luận. Lập luận kiểu tác tử thường kéo dài qua nhiều vòng tương tác, tạo ra nhu cầu còn lớn hơn về cửa sổ ngữ cảnh dài.

Nhu cầu này đã trực tiếp thúc đẩy ngành công nghiệp LLM phát triển các mô hình hỗ trợ cửa sổ ngữ cảnh khổng lồ. Chẳng hạn, nhiều LLM hiện hỗ trợ độ dài ngữ cảnh tới 128K token, và các mô hình tiên tiến hơn, như Claude 3.5 của Anthropic [Anthropic, 2024] và Gemini 1.5 của Google [Google, 2024], hỗ trợ hàng trăm nghìn token, thậm chí một triệu token. Bạn đọc quan tâm có thể tham khảo Chương 6 và Chương 8 để có thảo luận chi tiết hơn về các LLM ngữ cảnh dài.

Tuy nhiên, xử lý ngữ cảnh dài cho lập luận dẫn tới những thách thức mới vượt ra ngoài việc nén ngữ cảnh thông thường. Trong các tác vụ như tóm tắt một cuốn tiểu thuyết dài hay trích xuất ý chính từ tài liệu, LLM có thể nén thông tin và bỏ qua các chi tiết nhỏ. Ngược lại, lập luận đòi hỏi tính liên tục logic chặt chẽ, độ chính xác cao. Mỗi bước trung gian là một mắt xích quan trọng trong một chuỗi phụ thuộc lẫn nhau rất mạnh. Nếu mô hình "quên" một giả thiết cụ thể được nêu trong prompt, hoặc mất dấu một biến đã tính ở bước 2 khi đang thực hiện bước 50, toàn bộ phép suy diễn sẽ sụp đổ. Vì vậy, co giãn ngữ cảnh cho lập luận không chỉ là mở rộng bộ nhớ đệm KV (*KV cache*) để chứa nhiều token hơn. Nó đòi hỏi mô hình truy xuất thông tin một cách đáng tin cậy qua những khoảng cách xa và duy trì tính nhất quán logic chặt chẽ.

12.5 Đánh giá các LLM lập luận

Trong mục này, chúng ta bàn về cách đánh giá các LLM lập luận. Việc này tương tự đánh giá các LLM tiêu chuẩn [Chang và cộng sự, 2024b]. Tuy nhiên, các bài toán lập luận phức tạp đến mức ta thường cần đánh giá chi tiết hơn cùng những cân nhắc đặc biệt khác.

12.5.1 Đánh giá dựa trên kết quả và đánh giá dựa trên quá trình

Khi đánh giá một LLM lập luận tốt đến đâu, phương pháp dễ nhất và phổ biến nhất là **đánh giá dựa trên kết quả** (*outcome-based evaluation*). Trong cách này, ta chỉ nhìn vào câu trả lời cuối cùng mà mô hình tạo ra. Nếu câu trả lời cuối cùng khớp với đáp án chuẩn, mô hình được xem là đúng. Phương pháp này được dùng rộng rãi trong các bộ đánh giá chuẩn (*benchmark*) lập luận phổ biến, như GSM8K [Cobbe và cộng sự, 2021] và MATH [Hendrycks và cộng sự, 2021b]. Tuy nhiên, đánh giá dựa trên kết quả gặp vấn đề "đúng đáp án nhưng sai lý do", còn gọi là dương tính giả (*false positive*). Một mô hình có thể sinh ra các bước logic sai sót nhưng tình cờ đi tới con số cuối cùng đúng. Nếu chỉ kiểm tra kết quả cuối cùng, ta có thể kết luận sai rằng mô hình có năng lực lập luận mạnh trong khi nó chỉ gặp may.

Để đánh giá chi tiết hơn, ta có thể dùng **đánh giá dựa trên quá trình** (*process-based evaluation*). Thay vì chỉ kiểm tra câu trả lời cuối cùng, ta có thể xem xét toàn bộ đường lập luận từng bước [Wang và cộng sự, 2024c; Xia và cộng sự, 2025]. Một mô hình chỉ được xem là đúng nếu cả logic trung gian lẫn kết luận cuối cùng đều chặt chẽ. Đây là một phương pháp lý tưởng nhưng rất khó thực hiện trong thực tế. Thách thức chính là "dữ liệu vàng" ở cấp bước rất khan hiếm và tốn kém để tạo ra. Hơn nữa, với bất kỳ bài toán phức tạp nào, số đường lập luận hợp lệ lớn theo hàm mũ. Không thể liệt kê thủ công mọi bước trung gian đúng cho mọi đường mà một mô hình có thể đi.

Để đánh giá dựa trên quá trình thực tế hơn, ta có thể thu hẹp phạm vi để đánh giá các quyết định lập luận cục bộ. Một cách là quy bài toán về một tác vụ đánh giá bước tiếp theo đơn giản: cho một chuỗi các bước trước đó đúng, ta kiểm tra xem mô hình lập luận có chọn hoặc sinh đúng bước logic tiếp theo hay không. Để làm điều này, có thể dùng trực tiếp các PRM để đánh giá chất lượng mỗi bước do LLM sinh ra. Phương pháp này cũng liên quan đến việc đánh giá mô hình phần thưởng [Lambert và cộng sự, 2025]: thay vì xem xét đầu ra của mô hình lập luận, ta đánh giá chất lượng của các PRM. Ngoài ra, ta có thể xét các năng lực nền tảng của mô hình lập luận, chẳng hạn khả năng phát hiện lỗi trong một đường lập luận cho trước. Điều này cũng có thể được xem là một phép thử năng lực siêu lập luận (*meta-reasoning*): ta yêu cầu mô hình không chỉ lập luận, mà còn đánh giá chính quá trình lập luận [Zeng và cộng sự, 2024].

Trong thực tế, đánh giá dựa trên quá trình không dựa trên một tiêu chuẩn duy nhất. Ta có thể đánh giá chất lượng của một bước lập luận từ nhiều góc độ khác nhau, như tính nhất quán logic, tính mạch lạc và tính khách quan [Lee và Hockenmaier, 2025]. Tuy nhiên, dù đánh giá dựa trên quá trình chi tiết hơn nhiều, nó đi kèm chi phí gán nhãn cao hơn nhiều, và thường chưa đủ vững chắc. Vì vậy, phần lớn nghiên cứu thực tế vẫn dựa vào đánh giá dựa trên kết quả đơn giản hơn.

12.5.2 Các độ đo đánh giá

Dưới đây chúng tôi liệt kê một số độ đo dùng để đánh giá các LLM lập luận.

- **Khớp chính xác và độ chính xác** (*Exact Match and Accuracy*):

Đây là độ đo đơn giản nhất và được dùng rộng rãi nhất cho các tác vụ có đáp án cuối cùng xác định, như các bài toán đố. Giả sử ta có một tập dữ liệu $\mathcal{D}$ gồm N bài toán. Với bài thứ i, ta trích câu trả lời cuối cùng $\hat{a}_i$ từ đường lập luận được sinh ra và so sánh với đáp án chuẩn a_i^*. Độ chính xác được định nghĩa là:

$$\text{Accuracy} \;=\; \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(\hat{a}_i = a_i^*). \tag{12.30}$$

Trong thực tế, phương pháp khớp chính xác (kiểm tra hai chuỗi có giống hệt nhau không) có thể quá nghiêm ngặt, chẳng hạn ta sẽ loại "0.5" khi đáp án chuẩn là "1/2". Vì vậy, các nhà nghiên cứu thường áp dụng các đoạn mã dựa trên quy tắc để chuẩn hóa câu trả lời về một định dạng chính tắc trước khi so sánh.

- **Pass@k**: Vì LLM mang tính xác suất, một lần giải mã duy nhất có thể thất bại trong lập luận chỉ do một lỗi nhỏ. Pass@k đo xác suất mô hình tìm được ít nhất một câu trả lời đúng nếu được phép sinh k đường lập luận độc lập [Chen và cộng sự, 2021]. Độ đo này đặc biệt phổ biến trong sinh mã và các tác vụ toán phức tạp. Với bài thứ i trong $\mathcal{D}$, nếu ta lấy mẫu $N_s \geq k$ đường và có C_i đường chứa câu trả lời đúng, điểm Pass@k được định nghĩa là:

$$\text{Pass@}k \;=\; \frac{1}{N}\sum_{i=1}^{N} \left(1 - \frac{\binom{N_s - C_i}{k}}{\binom{N_s}{k}}\right). \tag{12.31}$$

Để sinh nhiều đường lập luận khác nhau, ta có thể dùng các phương pháp lấy mẫu mô tả ở Mục 12.3.2.

Chú giải của người dịch: Hàm chỉ thị và công thức Pass@k

$\mathbf{1}(\cdot)$ là *hàm chỉ thị*: bằng 1 nếu điều kiện trong ngoặc đúng, bằng 0 nếu sai; nên (12.30) chỉ là "số bài làm đúng chia tổng số bài". Công thức (12.31) dùng tổ hợp lớp 11: $\binom{N_s}{k}$ là số cách chọn k lời giải từ N_s lời giải đã sinh; $\binom{N_s - C_i}{k}$ là số cách chọn mà *toàn bộ* k lời giải đều sai. Tỉ số là xác suất "chọn k cái đều trượt", lấy 1 trừ đi được xác suất "có ít nhất một cái đúng" (phép lấy biến cố đối). Ví dụ $N_s = 5$, $C_i = 2$, $k = 2$: xác suất đều sai là $\binom{3}{2}/\binom{5}{2} = 3/10$, vậy Pass@2 của bài này là 0.7.

- **Cons@k (Consensus@k, đồng thuận@k)**: Khác với Pass@k, vốn cần một "nhà tiên tri" (*oracle*) để kiểm chứng xem mỗi câu trả lời trong k câu được lấy mẫu có đúng không, Cons@k xây dựng một câu trả lời đồng thuận mà không cần xác định tính đúng sai của từng mẫu. Cụ thể, ta sinh k đường lập luận, trích các câu trả lời cuối cùng của chúng, rồi chọn câu trả lời xuất hiện nhiều nhất bằng bỏ phiếu đa số (*majority voting*) [Wang và cộng sự, 2023c]. Câu trả lời đồng thuận $\hat{a}_i^{\text{maj}}$ cho bài thứ i được định nghĩa là:

$$\hat{a}_i^{\text{maj}} \;=\; \arg\max_{a_i} \sum_{j=1}^{k} \mathbf{1}(\hat{a}_i^j = a_i), \tag{12.32}$$

  trong đó $\hat{a}_i^j$ là câu trả lời cuối cùng được trích từ đường lập luận thứ j. Độ chính xác cuối cùng sau đó nhận được bằng cách so sánh $\hat{a}_{\text{maj}}$ với đáp án chuẩn:

$$\text{Cons@}k \;=\; \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}(\hat{a}_i^{\text{maj}} = a_i^*). \tag{12.33}$$

  Cons@k cung cấp một cách đánh giá thực tế cho các hệ thống lập luận ngoài đời thực, vì nó đo xem mô hình có *nhất quán* đi tới câu trả lời đúng hay không. So với Pass@k, vốn đánh giá xem có ít nhất một phản hồi được lấy mẫu là đúng hay không, Cons@k nhấn mạnh độ tin cậy của lập luận bằng cách ưu tiên những câu trả lời được ủng hộ một cách nhất quán qua nhiều đường lập luận được lấy mẫu.

- **LLM làm giám khảo** (*LLM-as-a-Judge*): Với các tác vụ lập luận mở, như lập kế hoạch kiểu tác tử hay các câu hỏi phức tạp miền mở, việc khớp chính xác dựa trên quy tắc thường thất bại. Trong những trường hợp đó, ta có thể dùng LLM làm giám khảo tự động. Chẳng hạn, có thể dùng prompt yêu cầu một LLM giám khảo đánh giá các bước trung gian được sinh ra và chấm điểm chúng dựa trên những tiêu chí định sẵn. Một cách khác, có thể dùng LLM để so sánh hai mô hình lập luận. Giám khảo được xem đầu ra của cả hai mô hình cho cùng một truy vấn và quyết định đường lập luận nào tốt hơn, hoặc hai bên hòa. Tỉ lệ thắng (*win rate*) khi đó phản ánh tỉ lệ phần trăm số lần một mô hình cụ thể được ưa chuộng hơn.

Chú giải của người dịch: Pass@k hay Cons@k?

Giả sử với một bài toán, mô hình sinh $k = 5$ lời giải cho các đáp số $12, 15, 12, 7, 12$, và đáp án đúng là 15. Theo Pass@5, bài này được tính là *đạt* (vì có một lời giải ra 15). Theo Cons@5, đáp án đồng thuận là 12 (xuất hiện 3 lần) nên bài này *không đạt*. Pass@k giống như "được thi k lần, chỉ cần đỗ một lần"; còn Cons@k giống như "nộp đáp án mà đa số các lần làm cho ra" — điều này sát với thực tế hơn khi ta không có sẵn đáp án để kiểm tra.

12.5.3 Các vấn đề còn bỏ ngỏ

Một vấn đề là nhiễm bẩn dữ liệu (*data contamination*). Khi các mô hình ngôn ngữ được huấn luyện trên những tập dữ liệu ngày càng khổng lồ thu thập từ internet, việc giữ dữ liệu kiểm tra nằm ngoài quá trình huấn luyện trở nên rất khó. Nếu một mô hình đã thấy các câu hỏi của một bộ đánh giá chuẩn lập luận (như MATH hay GSM8K) trong giai đoạn tiền huấn luyện, nó có thể chỉ đơn giản đọc lại một lời giải đã học thuộc thay vì thực sự lập luận tại chỗ. Điều này khiến khó xác định điểm số cao trên bộ đánh giá chuẩn phản ánh năng lực giải quyết vấn đề thực sự hay chỉ là trí nhớ tốt. Để giải quyết vấn đề này, cộng đồng được khuyến khích khám phá những bộ đánh giá chuẩn mới mà các bài toán được cập nhật liên tục, cũng như các tập dữ liệu giữ riêng không bao giờ công bố.

Một vấn đề khác là độ vững (*robustness*). Các mô hình lập luận hiện nay đôi khi rất mong manh. Một mô hình có thể giải hoàn hảo một bài toán phức tạp, nhưng thất bại hoàn toàn nếu chính câu hỏi đó được diễn đạt lại một chút hoặc có thêm các chi tiết không liên quan vào prompt. Sự nhạy cảm này gợi ý rằng mô hình đôi khi dựa vào các khuôn mẫu văn bản bề mặt thay vì thực sự hiểu logic bên dưới. Để lập luận vững hơn, ta cần các LLM dễ dàng thích ứng với những tác vụ mới, thậm chí khó hơn.

Hơn nữa, LLM có thể sinh ra các đường lập luận bị ảo giác (*hallucination*) hoặc không trung thực (*unfaithful*). Ví dụ, chúng có thể bịa ra những quy tắc toán học giả để đi tới kết luận. Trong một số trường hợp, văn bản được sinh ra trông như một phép suy diễn logic hoàn hảo, nhưng thực ra mô hình đi tới câu trả lời qua một cơ chế ẩn (như đoán may hay một lối tắt đã học thuộc). Sự không trung thực này cho thấy một khoảng cách giữa văn bản mô hình xuất ra và các biểu diễn nội tại

thực sự của nó. Trong các thiết lập học tăng cường, điều này thường liên quan đến vấn đề lách phần thưởng (*reward hacking*). Nếu mô hình chỉ được thưởng cho câu trả lời cuối cùng, chúng thường học cách "gian lận" hệ thống phần thưởng bằng logic sai sót. Vì vậy, việc đưa giám sát từng bước vào học tăng cường, như giám sát quá trình, là rất có giá trị để kiểm tra xem mỗi bước trung gian có thực sự đúng hay không.

Vấn đề thứ tư là bão hòa bộ đánh giá chuẩn (*benchmark saturation*). Tiến bộ nhanh chóng của các LLM lập luận khiến các bộ đánh giá chuẩn thông thường nhanh chóng trở nên lỗi thời. Các mô hình hiện đạt điểm gần như tuyệt đối trên những bài kiểm tra mà chỉ một hai năm trước còn được xem là cực kỳ khó. Sự bão hòa này khiến việc so sánh các mô hình mới hay đo tiến bộ thực sự của lĩnh vực trở nên khó khăn. Vì vậy, các nhà nghiên cứu cần phát triển những bài kiểm tra khó hơn để đánh giá năng lực suy diễn logic thực sự trên các bài toán mới. Lưu ý rằng lập luận ở mức con người thường mang tính mở và tương tác. Một cách khác để kiểm tra các mô hình lập luận có thể là dùng chúng trong các tác tử tự hành, vốn có thể khám phá, dùng công cụ bên ngoài, và tự sửa sai.

12.6 Bàn thêm

Bây giờ chúng ta bàn về một số vấn đề thú vị liên quan đến các LLM lập luận.

12.6.1 Lập luận bên ngoài và lập luận bên trong

Trong các thảo luận trước, ta đã giả định rằng quá trình lập luận có thể được nắm bắt tốt bằng một đường lập luận dạng văn bản (ví dụ CoT). Điều này đặt ra một câu hỏi căn bản: Văn bản CoT được sinh ra có thực sự phản ánh cách mô hình lập luận hay không? Câu hỏi này dẫn ta tới việc xem xét sự khác biệt giữa lập luận được *ngoại hiện* ra văn bản và quá trình "suy nghĩ" thực sự bên trong.

Khi ta yêu cầu một LLM suy nghĩ từng bước, ta đang buộc nó ngoại hiện quá trình tính toán thành các token rời rạc. Tuy nhiên, ta không thể đơn giản xem chuỗi văn bản này là quá trình lập luận thực sự. Một mặt, một LLM có thể sinh ra một CoT chi tiết và chặt chẽ về logic, nhưng việc

ra quyết định thực sự có thể đã diễn ra ngầm trong các biểu diễn ẩn của ngay vài token đầu tiên. Mặt khác, một mô hình có thể tạo ra một CoT sai sót hoặc vô nghĩa nhưng vẫn xuất ra câu trả lời cuối cùng đúng, cho thấy các biểu diễn bên trong của nó đang theo dõi tác vụ theo cách khác với những gì văn bản được sinh ra gợi ý.

Các nhà nghiên cứu đã chỉ ra rằng các lời giải thích CoT có thể gây hiểu lầm [Turpin và cộng sự, 2023; Chen và cộng sự, 2025]. Ví dụ, LLM có thể có thiên hướng sinh một CoT nghe rất logic để đi tới một câu trả lời sai, dù câu trả lời đúng đã được nhắc đến trong prompt. Để khám phá LLM thực sự suy nghĩ như thế nào, các nhà nghiên cứu chuyển sang phân tích kích hoạt nơ-ron và khả năng diễn giải cơ chế (*mechanistic interpretability*). Mục tiêu là hiểu điều gì xảy ra bên trong các tầng ẩn của một LLM trong quá trình sinh. Ví dụ, trong công trình của Fraser-Taliente và cộng sự [2026], các bộ tự mã hóa ngôn ngữ tự nhiên (*natural language auto-encoder*) được dùng để chuyển các giá trị kích hoạt thành văn bản ngôn ngữ tự nhiên. Một bộ tự mã hóa ngôn ngữ tự nhiên gồm một *bộ diễn đạt kích hoạt* (*activation verbalizer*) và một *bộ tái tạo kích hoạt* (*activation reconstructor*), được dùng để ánh xạ trực tiếp các giá trị kích hoạt trong dòng residual (*residual stream*) bên trong thành văn bản con người đọc được và ngược lại. Bằng cách này, ta có thể giải mã các giá trị kích hoạt để "đọc" suy nghĩ của LLM.

Một hướng nghiên cứu khác tập trung vào lập luận không cần văn bản, gọi là lập luận tiềm ẩn (*latent reasoning*) hay lập luận ngầm (*implicit reasoning*). Trong lập luận tiêu chuẩn, như ta đã định nghĩa, một đường lập luận gồm một chuỗi các bước bằng văn bản. Trong lập luận tiềm ẩn, quá trình này diễn ra bên trong trước khi mô hình "nói". Điều này đạt được hoặc bằng cách thay các token rời rạc bằng các vectơ liên tục, hoặc bằng cách sinh ra các từ văn bản thực sự "ở hậu trường" và giấu chúng khỏi người dùng. Dưới đây chúng tôi trình bày một số phương pháp như vậy.

- Một trong những ý tưởng đơn giản nhất là dùng "token tạm dừng (*pause token*)". Như thường lệ trong sinh tự hồi quy, mô hình ngôn ngữ có một bước tính toán nội bộ cho mỗi từ nó xuất ra. Điều thú vị là người ta phát hiện rằng nếu ta đưa vào mô hình các token giả (gọi là token tạm dừng), mô hình có thêm thời gian tính toán để xử lý thông tin trong các tầng ẩn trước khi phải xuất ra câu trả lời cuối cùng [Goyal và cộng sự, 2024]. Điều này cho thấy LLM có thể thực

hiện những tính toán "thầm lặng", hay suy nghĩ tiềm ẩn, mà không cần sinh ra từ ngữ thực sự.

- Một cách tiếp cận khác dùng chưng cất tri thức để nén lập luận tường minh vào các trạng thái ẩn. Chẳng hạn, phương pháp *chuỗi suy nghĩ ngầm* (*implicit chain of thought*) huấn luyện một mô hình học sinh bắt chước trực tiếp các trạng thái ẩn bên trong của một mô hình giáo viên lớn hơn đang "nghĩ thành lời" [Deng và cộng sự, 2023]. Kết quả là mô hình học sinh học được cách bỏ qua việc sinh văn bản. Ngoài ra, LLM cũng có thể tự học từ chính nó bằng các kỹ thuật tự chưng cất (*self-distillation*) [Shen và cộng sự, 2025]: ta có thể buộc quá trình suy nghĩ tiềm ẩn khớp với các trạng thái ẩn cuối cùng của lập luận dạng văn bản. Trong cả hai phương pháp, suy nghĩ tiềm ẩn được hiện thực hóa như một không gian toán học liên tục: thay vì sinh một chuỗi văn bản đọc được từng bước, mô hình tính các bước lập luận bằng các biểu diễn ẩn dạng vectơ.

- Một phương pháp khác để lập luận trong không gian tiềm ẩn liên tục là dùng mô thức **chuỗi suy nghĩ liên tục** (*chain of continuous thought*) (**COCONUT**) [Hao và cộng sự, 2025]. Trong sinh văn bản tiêu chuẩn, đầu ra của tầng cuối được chuyển thành một token rời rạc, rồi token đó được đưa trở lại tầng đầu tiên cho bước tiếp theo. Quá trình này buộc mô hình nén tính toán bên trong của mình vào một token duy nhất. COCONUT bỏ qua bước này trong giai đoạn lập luận. Nó lấy vectơ đầu ra của tầng cuối và đưa thẳng trở lại tầng đầu vào như một *suy nghĩ liên tục*. Bằng cách lặp lại quá trình này, mô hình xây dựng một chuỗi các vectơ liên tục trước khi bắt đầu sinh câu trả lời văn bản cuối cùng. Vì vectơ liên tục có thể chứa nhiều thông tin hơn token rời rạc, mô hình có thể khám phá nhiều đường lập luận cùng lúc mà không phải cam kết sớm vào một lựa chọn token cụ thể. Cách tiếp cận này cho thấy LLM không nhất thiết phải dựa vào ngôn ngữ của con người để giải các bài toán phức tạp. Bằng cách giữ quá trình lập luận trong một không gian liên tục, chúng có thể tính ra câu trả lời hiệu quả hơn.

- Thay vì lập luận trong không gian liên tục, ta cũng có thể dùng các token văn bản bình thường để biểu diễn suy nghĩ bên trong. Ví dụ, trong phương pháp Quiet-STaR [Zelikman và cộng sự, 2024], mô

hình được huấn luyện để "nghĩ thầm" ngay giữa lúc đang đọc hay viết văn bản bình thường. Ý tưởng rất đơn giản: trước khi dự đoán token tiếp theo, mô hình sinh ra một chuỗi ngắn các token "suy nghĩ" ẩn. Bạn có thể hình dung điều này như một độc thoại nội tâm hay một tờ nháp nhanh trong đầu mà người dùng không bao giờ thấy. Sau khi sinh ra các suy nghĩ ẩn này, LLM dùng chúng để hỗ trợ đoán từ tiếp theo thực sự. Nếu các suy nghĩ này giúp LLM dự đoán đúng từ, LLM được thưởng và học được rằng cách nghĩ này hữu ích. Nếu các suy nghĩ vô ích hoặc gây rối, LLM học cách loại bỏ chúng.

Chú giải của người dịch: Token rời rạc và vectơ liên tục

Khi chọn một token, mô hình phải "chốt" một từ duy nhất trong bộ từ vựng, giống như làm tròn số $3{,}14159\ldots$ thành 3 — phần thông tin còn lại bị mất. Vectơ ở tầng cuối thì giống như con số chưa làm tròn: nó có thể đồng thời "nghiêng" về vài khả năng (chẳng hạn 60% hướng A, 40% hướng B). COCONUT giữ nguyên vectơ chưa làm tròn đó làm đầu vào cho bước sau, nên không mất thông tin, nhưng đổi lại con người không đọc được các "suy nghĩ" này.

12.6.2 Ghi nhớ và lập luận

Khi bàn về LLM, một câu hỏi thường được đặt ra: Mô hình có thực sự "suy nghĩ", hay nó chỉ có một bộ nhớ lớn và đang đọc lại một câu trả lời tương tự từng thấy trong dữ liệu huấn luyện? Điều này dẫn tới cuộc thảo luận so sánh lập luận và ghi nhớ (*memorization*) trong LLM.

Nói đơn giản, sự khác biệt giữa hai điều này có thể tóm tắt như sau:

- **Ghi nhớ là nhìn về phía sau.** Mô hình điều chỉnh tham số của nó để học các khuôn mẫu từ dữ liệu huấn luyện. Khi gặp một đầu vào tương tự, mô hình chỉ đơn giản "phát lại" các khuôn mẫu đã học.
- **Lập luận là tiến về phía trước.** Thay vì tìm một câu trả lời làm sẵn, mô hình áp dụng các quy tắc và logic tổng quát để xây dựng một chuỗi suy nghĩ từng bước. Cốt lõi của lập luận là *khả năng tổng quát hóa*: kể cả khi đối mặt với một bài toán hoàn toàn mới chưa từng xuất hiện trong tập huấn luyện, mô hình vẫn có thể giải nó bằng các công cụ logic.

Trong những nghiên cứu học sâu thời kỳ đầu, ranh giới này không phải lúc nào cũng rõ ràng. Ví dụ, Zhang và cộng sự [2021b] chỉ ra rằng các mạng nơ-ron sâu có năng lực ghi nhớ khổng lồ, dễ dàng học thuộc cả dữ liệu nhiễu với nhãn hoàn toàn ngẫu nhiên. Với LLM, do kích thước mô hình rất lớn, chúng không chỉ có thể ghi nhớ Wikipedia mà cả những chi tiết ẩn sâu trong các kho mã nguồn.

Khả năng ghi nhớ này thường "đội lốt" lập luận. Ví dụ, nếu ta đưa cho một LLM một bài toán thuật toán kinh điển nhưng sửa nhẹ các con số, LLM có thể dựa vào "quán tính trí nhớ" và vẫn xuất ra câu trả lời gốc, do đó mắc một lỗi cơ bản. Hiện tượng này thường được gọi là "con vẹt ngẫu nhiên (*stochastic parrots*)" [Bender và cộng sự, 2021], nghĩa là LLM bắt chước một cách mù quáng và chắp vá các đoạn văn bản đã ghi nhớ mà không thực sự hiểu logic bên dưới.

Để thấy rõ hơn sự khác biệt giữa ghi nhớ và lập luận, ta có thể xem xét hai khía cạnh.

1. Hiện tượng grokking và khả năng tổng quát hóa

Khi tìm hiểu cách mô hình chuyển từ ghi nhớ sang hiểu thực sự, các nhà nghiên cứu đã có một phát hiện thú vị gọi là "grokking (*grokking*)" [Power và cộng sự, 2022]. Họ huấn luyện một mô hình Transformer nhỏ học các phép toán hai ngôi trong đại số trừu tượng (như phép cộng modulo). Họ nhận thấy một hiện tượng rất thú vị: ở đầu quá trình huấn luyện, mô hình nhanh chóng đạt độ chính xác 100% trên tập huấn luyện, nhưng độ chính xác trên tập kiểm tra gần như bằng 0. Ở giai đoạn này, rõ ràng mô hình chỉ đang học vẹt các đẳng thức. Tuy nhiên, nếu không dừng huấn luyện mà để mô hình tiếp tục lặp qua dữ liệu rất lâu sau thời điểm quá khớp (*overfitting*), một điều bất ngờ xảy ra. Đột nhiên, độ chính xác trên tập kiểm tra vọt lên 100% trong một thời gian rất ngắn. Mô hình dường như trải qua một "khoảnh khắc à há".

Các nghiên cứu sâu hơn cho thấy trước "khoảnh khắc à há" này, tham số mô hình đang ghi nhớ một cách lộn xộn các đẳng thức cụ thể (như $2 + 3 = 5$) [Nanda và cộng sự, 2023]. Nhưng khi grokking xảy ra, bên trong mạng hình thành một biểu diễn toán học tổng quát dùng các hàm lượng giác và biến đổi Fourier (*Fourier transform*). Nó đã tìm ra một thuật toán phổ quát có thể giải thích toàn bộ dữ liệu.

Chú giải của người dịch: Phép cộng modulo và lượng giác

Phép cộng modulo p là "cộng rồi lấy số dư khi chia cho p", giống như cộng giờ trên mặt đồng hồ: 9 giờ $+5$ giờ $=$ 2 giờ (modulo 12). Nếu đặt các số $0, 1, \ldots, p-1$ thành các điểm trên đường tròn lượng giác, ứng với góc $2\pi a/p$, thì phép cộng modulo chính là *cộng góc*, và có thể tính bằng công thức lớp 11 $\cos(\alpha+\beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$. Mạng nơ-ron sau khi grokking đã tự "khám phá" ra cách làm gần giống như vậy, thay vì học thuộc bảng cộng.

Thí nghiệm này cho ta biết lập luận không tự nhiên xuất hiện từ hư không. Nó thường nổi lên như một trạng thái tổng quát hóa ở mức cao khi việc ghi nhớ chạm tới nút thắt, hoặc sau quá trình huấn luyện kéo dài, khi mạng tổ chức lại các tham số của mình để tìm ra lời giải thích phổ quát nhất.

2. Đánh đổi về chi phí

Từ góc nhìn của lý thuyết thông tin và hiệu quả tính toán, LLM tự nhiên cân bằng giữa ghi nhớ và lập luận dựa trên chi phí. Một mặt, ghi nhớ đòi hỏi nhiều dung lượng tham số hơn. Để ghi nhớ mọi sự kiện trên thế giới, kích thước mô hình sẽ phải tăng vô hạn. Mặt khác, lập luận đòi hỏi nhiều bước tính toán hơn lúc suy luận. Chỉ cần mô hình ghi nhớ các quy tắc tổng quát của phép cộng và phép nhân, nó có thể giải nhiều bài toán bằng cách thực hiện các bước tính toán như vậy trong lúc sinh.

Vì vậy, LLM thường đi theo "con đường ít trở ngại nhất". Với các sự kiện phổ biến, truy xuất câu trả lời trực tiếp từ tham số (hay bộ nhớ) là cách nhanh nhất. Nhưng với các tác vụ hiếm gặp, nhiều bước, dùng lối nghĩ Hệ thống 2 là cách phù hợp hơn.

Điều này cũng giải thích vì sao các kỹ thuật như sinh văn bản tăng cường truy xuất (*retrieval-augmented generation*) (RAG) trở nên phổ biến. Bản chất của RAG là chuyển gánh nặng ghi nhớ sang một cơ sở dữ liệu bên ngoài, nhờ đó ta có thể giải phóng không gian tham số cho lập luận. Bằng cách tách việc truy xuất tri thức khỏi suy diễn logic, LLM không còn cần ghi nhớ những sự kiện bên ngoài liên tục thay đổi. Thay vào đó, nó biết cách tra cứu và xử lý chúng.

12.6.3 LLM có thực sự lập luận được không?

Câu hỏi liệu LLM có thực sự lập luận được hay không là một trong những chủ đề gây tranh cãi nhất trong AI hiện nay. Khi thấy một LLM giải hoàn hảo một bài toán hay lập một lịch trình phức tạp, ta rất dễ tin rằng nó hiểu thế giới giống như con người. Tuy nhiên, để trả lời câu hỏi này, ta phải xem các mô hình này thực sự hoạt động như thế nào.

Về bản chất, LLM hoạt động bằng cách dự đoán từ tiếp theo. Vì đã xử lý một lượng văn bản khổng lồ, các mô hình này nắm vững hoàn toàn các quy tắc, ngữ pháp và cấu trúc của ngôn ngữ con người. Trong khoa học nhận thức, điều này được gọi là năng lực ngôn ngữ hình thức (*formal linguistic competence*) [Chomsky, 1965; Mahowald và cộng sự, 2024]. Chúng biết chính xác các từ nên được sắp xếp theo thứ tự hợp lý như thế nào. Tuy nhiên, chúng gặp khó khăn với năng lực ngôn ngữ chức năng (*functional linguistic competence*), tức là khả năng dùng ngôn ngữ để hiểu và xoay xở trong thế giới thực [Kambhampati và cộng sự, 2024]. Khác với con người, vốn học bằng cách tương tác vật lý với môi trường, LLM chỉ biết các khuôn mẫu ẩn của văn bản. Vì không được "neo" vào thế giới thực, chúng thiếu hiểu biết thực sự về không gian vật lý và quan hệ nhân–quả chặt chẽ [Mahowald và cộng sự, 2024].

Hạn chế này trở nên rõ rệt khi ta kiểm tra LLM trên các tác vụ lập kế hoạch [Valmeekam và cộng sự, 2023]. Ví dụ, với các bài toán lập kế hoạch kinh điển, như di chuyển các khối màu trên bàn để xếp thành một chồng cụ thể, LLM thường giải được nếu bài toán dùng những từ thông dụng mà nó đã thấy trong dữ liệu huấn luyện. Nhưng nếu tên các khối và các hành động được thay bằng những từ vô nghĩa, hiệu năng giảm mạnh [Kambhampati, 2024]. Một hệ thống lập luận thực sự sẽ không quan tâm đến tên gọi, vì logic toán học bên dưới hoàn toàn giống nhau. LLM thất bại vì khuôn mẫu văn bản quen thuộc của nó bị phá vỡ.

Hơn nữa, các kỹ thuật như prompt CoT buộc mô hình xuất ra một quá trình từng bước. Điều này thể hiện phần nào hành vi suy nghĩ chậm rãi, có cân nhắc của con người. Nhưng thực ra, LLM vẫn chỉ đang tiến thẳng về phía trước một cách nghiêm ngặt và dự đoán xem một lời giải thích logic *thường trông như thế nào*. Nếu LLM mắc lỗi hoặc mang một giả định thiên lệch, nó thường tự tin sinh ra một lời giải thích sai sót chỉ để biện minh cho câu trả lời sai của mình [Chen và cộng sự, 2025].

Rốt cuộc, để trả lời LLM có "thực sự" lập luận được hay không, trước hết ta cần định nghĩa lập luận thực sự nghĩa là gì. Nếu ta kỳ vọng LLM suy nghĩ giống hệt con người, thì từ thảo luận trên rõ ràng có những khác biệt. Tuy nhiên, nếu xem lập luận thuần túy như một quá trình chức năng, câu trả lời sẽ khác. Lập luận có thể được định nghĩa đơn giản là khả năng tiếp nhận một bài toán phức tạp, chia nhỏ nó, áp dụng các quy tắc logic, và đi tới một kết luận đúng. Khi một LLM giải đúng một phương trình toán từng bước, hay khi nó viết một đoạn mã logic để xử lý dữ liệu, nó đang thực thi một chuỗi logic hợp lệ. Ngay cả khi khả năng này nảy sinh từ việc huấn luyện các mạng nơ-ron quy mô lớn chứ không phải từ nhận thức giống con người, kết quả vẫn là giải quyết vấn đề thành công. Khi LLM ngày càng lớn hơn và học từ nhiều dữ liệu hơn, chúng vượt ra khỏi việc chỉ ghi nhớ câu trả lời và bắt đầu nắm bắt các quy tắc logic bên dưới.

12.7 Tóm tắt

Trong chương này, chúng ta đã trình bày cách áp dụng LLM vào lập luận, vốn được xem là một trong những tác vụ thách thức nhất trong NLP. Cụ thể, chúng ta đã bàn về các phương pháp cải thiện khả năng lập luận trong giai đoạn huấn luyện và giai đoạn suy luận. Chúng ta cũng đã tìm hiểu các phương pháp đánh giá LLM lập luận, cùng một số vấn đề còn bỏ ngỏ.

Phát hiện rằng LLM có thể vạch ra tường minh các bước lập luận của mình đã truyền cảm hứng cho những hướng nghiên cứu và ứng dụng thực tiễn mới. Thay vì chỉ xem lập luận là một cách để trả lời các câu hỏi riêng lẻ, nghiên cứu hiện nay xem lập luận như một công cụ thúc đẩy những hành vi phức tạp và tự chủ hơn.

Thứ nhất, lập luận gắn chặt với khái niệm "suy nghĩ dài", điều làm thay đổi căn bản cách sử dụng năng lực tính toán trong LLM. Như đã bàn ở Mục 12.3, bằng cách cho mô hình nhiều thời gian và năng lực tính toán hơn để suy nghĩ, chúng có thể giải những bài toán khó hơn nhiều. Điều này mang lại một thay đổi lớn trong cách LLM hoạt động: giờ đây ta cần tìm cách phân bổ năng lực tính toán tốt nhất trong lúc mô hình đang sinh câu trả lời, chứ không chỉ trong lúc huấn luyện ban đầu. Giải quyết vấn đề này ở cấp hệ thống nằm ngoài phạm vi cuốn sách. Tuy nhiên, xây dựng những mô hình lập luận hiệu quả cao, biết nên dành bao nhiêu thời gian để suy nghĩ, là một hướng nghiên cứu rất hứa hẹn và thiết thực.

Thứ hai, khả năng lập luận là nền tảng của lập luận kiểu tác tử (*agentic reasoning*), trong đó LLM thôi là những cỗ máy trả lời thụ động và trở thành những bên tham gia chủ động trong một môi trường. Trong sinh văn bản tiêu chuẩn, lập luận là một đường thẳng từ prompt tới câu trả lời. Ngược lại, lập luận kiểu tác tử vận hành trong một vòng lặp liên tục gồm suy nghĩ, hành động và quan sát [Yao và cộng sự, 2023]. Tuy nhiên, vòng lặp này dẫn tới những thách thức lớn. Vấn đề là LLM có thể dễ dàng bị lạc hoặc mắc kẹt trong vòng lặp. Chúng có thể lặp lại cùng một lỗi, lập kế hoạch tồi, hoặc không biết cách gượng dậy khi một hành động thất bại [Shinn và cộng sự, 2023]. Giữ cho mô hình đi đúng hướng trong thời gian dài vẫn còn rất khó.

Thứ ba, lập luận trực tiếp tạo điều kiện cho lập luận tăng cường công cụ (*tool-augmented reasoning*), giúp khắc phục những hạn chế của LLM, như không thể tính toán chính xác hay truy cập các sự kiện theo thời gian thực. Thay vì buộc LLM đoán một câu trả lời, ta có thể huấn luyện LLM nhận ra giới hạn của chính mình và viết mã [Gao và cộng sự, 2023b] hoặc gọi các API bên ngoài [Schick và cộng sự, 2024] để đảm nhận phần việc nặng. Tuy nhiên, vấn đề chính là dạy LLM khi nào cần một công cụ và chọn công cụ nào trong số rất nhiều công cụ [Qin và cộng sự, 2024]. Ngoài ra, nếu một công cụ hỏng hoặc trả về dữ liệu kỳ lạ, LLM phải đủ thông minh để phát hiện lỗi và thử lại, thay vì chỉ tin vào kết quả sai.

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.